

# ***Perifériás rendszerek és interfészek***

## ***Bevezető***

# Követelmények, pontozás

Join classroom

Automatika, Számítástechnika, Távközlés – III. év

| Labor | Szóbeli | Végső jegy |
|-------|---------|------------|
| 60%   | 40%     |            |

Mechatronika – IV. év

| Labor | Szóbeli | Végső jegy |
|-------|---------|------------|
| 50%   | 50%     |            |

**zqewfzj3**



## Jelenlét kötelező:

- Labor 100%
- Előadás 70%

# Előadások tartalma

| Hét     | Ea. | Dátum  | <u>Perifériák és interfészek - az előadások témája</u>                           |
|---------|-----|--------|----------------------------------------------------------------------------------|
| 1       | 1   | Feb 16 | Bevezető. Beágyazott rendszerek, párhuzamosítás, PicoBlaze, SoC                  |
|         | 2   | Feb 18 | A PC-k Ki/Bemeneti rendszere. Adatátviteli módszerek, sínek elektromos jellemzői |
| 2       | 3   | Feb 25 | Sínek csoportosítása, arbitrációs módszerek, VME, EISA, A PCI sín                |
| 3       | 4   | Mar 2  | Soros sínek. Az USB 2.0                                                          |
|         |     | Mar 4  |                                                                                  |
| 4       | 5   | Mar 11 | Soros sínek. Az USB 3.0, a FireWire sín és a PCI Express                         |
| 5       | 6   | Mar 16 | Grafikus interfészek. Az AGP illesztő.                                           |
|         | -   | Mar 18 | -                                                                                |
| 6       | 7   | Mar 25 | Képernyők (CRT és LCD, OLED) felépítése és működése                              |
| 7       | 8   | Mar 30 | Háttértárolók. SSDk, HDDk felépítése és működése. RAID rendszerek                |
|         | 9   | Apr 1  | Optikai háttértárolók. CDROM, DVD, Blue-ray                                      |
| Vakáció |     |        |                                                                                  |
| 8       | 10  | Apr 15 | Külső perifériák: Nyomtatók, Scanner-ek, Digitizálók, Vetítők                    |
| 9       | 11  | Apr 20 | Szenzor-interfészek: SPI, I <sup>2</sup> C, 1-Wire Bus                           |
|         | 12  | Apr 22 | Autóipari interfész-technika, CAN, LIN                                           |
| 10      | 13  | Apr 29 | IoT wireless protokollok: Bluetooth, WiFi, ZigBee                                |
| 11      | 14  | May 4  | Vizsga előtti konzultáció                                                        |

**Könyvészet:**

[https://opac3.ms.sapientia.ro/hu\\_HU/results/-/results/shared/e714ca51-1c83-4ad1-aab4-8dba4e08bcba](https://opac3.ms.sapientia.ro/hu_HU/results/-/results/shared/e714ca51-1c83-4ad1-aab4-8dba4e08bcba)



# ***Perifériás rendszerek és interfészek***

## *I. előadás*

- ***Beágyazott rendszerek – Embedded systems***
- ***Párhuzamosítási módszerek***
- ***A PicoBlaze beágyazott processzor***
- ***SoC – Hardware/Software particionálás***

# Beágyazott információs rendszerek:

---

- ❑ a befogadó fizikai/kémiai/biológiai környezetükkel intenzív, valós idejű információs kapcsolatban álló,
- ❑ autonóm működésű,
- ❑ szolgáltatás-biztos (dependable),
- ❑ “láthatatlan”
- ❑ IoT rendszerek
- ❑ SoC, NoC rendszerek

## Kiemelt feladatuk

- **a befogadó környezet identifikációja**  
folyamatos adatgyűjtés, valós idejű modellépítés
- **a befogadó környezet befolyásolása**  
valós idejű kiértékelés, döntés, beavatkozás

## **számítógépes rendszerek, melyek**

- lokálisan (általában) korlátozott,
- globálisan (általában) bőséges erőforrásokkal (idő, adat, tápellátás, memória, ...) rendelkeznek.

**Megoszlás: 10% PC  $\Leftrightarrow$  90% beágyazott rendszer !**

# Ambiens rendszerek:

---

Embedded Systems  
Pervasive Computing  
Ubiquitous Computing

Mindenütt jelenlevő  
számítástechnika/informatika

Ambient Intelligence

**Emberközpontú  
információtechnológia**

**Ambiens rendszerek:** az életkörülmények javítását célzó,  
emberközpontú számítástechnikai/informatikai rendszerek

# Beágyazott szoftver:

---

**Univerzális rendszerintegrátor:** a valós rendszerek alkotóelemei egyre inkább „számítástechnikai” kölcsönhatások révén működnek együtt.

(Audi 8: ~2000 jel, ~60 elektronikus kontroller (ECU))

*„... Software is Hard and Hardware is ...”*

## Következmények:

1. A szoftver egyre jobban az alkalmazási terület részévé válik.
2. A szoftvereknek a funkcionális követelmények mellett fizikai követelményeknek is eleget kell tenniük.

**Kérdés: Hogyan kell ilyen rendszereket tervezni?**

# Főbb technológiai területek és kihívások

---

- 1. A beágyazott rendszerek architektúrája:** megfelelő keret ahhoz, hogy előzetesen igazoltan helyes működésű részegységekből működőképes rendszer jöjjön létre miközben a komponensek “belsejét” nem ismerjük ...  
**„Plug-and-play”,** valós-idejű & szinkronizált működés, tápáram-felvétel minimalizálás ...
- 2. Rendszer tervezés:** (1) rendszer modellezés, (2) automatikus szintézis, (3) fejlesztő eszközök.
- 3. A fejlesztési eredmény helyességének igazolása:** igazolás és bizonylatolás (validation and certification).
- 4. Szolgáltatásbiztonság:** ~ autonóm működésű alrendszerek dinamikus újrakonfigurálása, biztonság, hibatűrés.

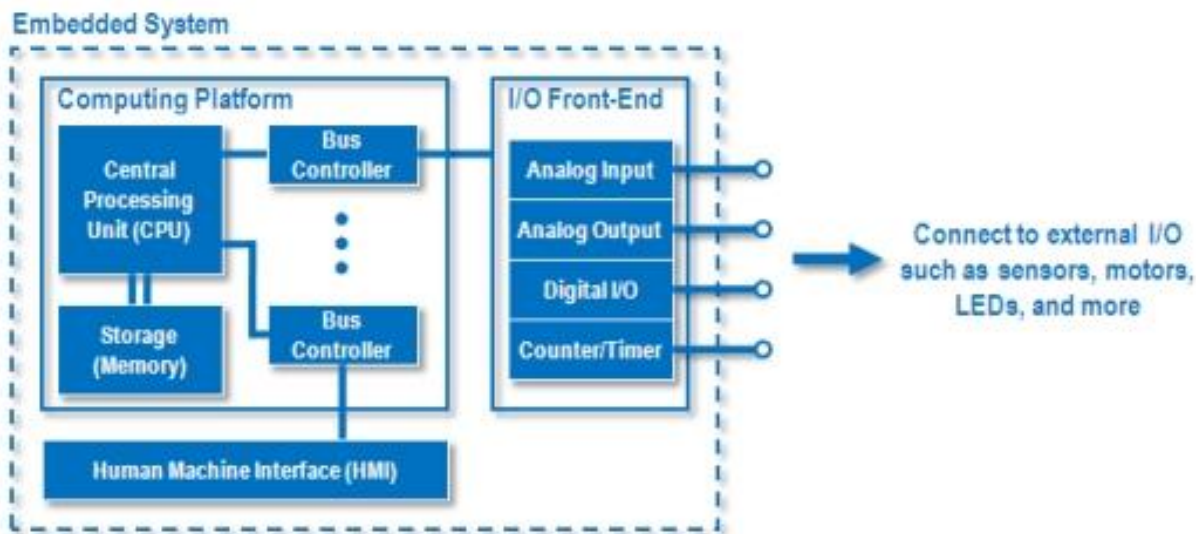


# Főbb technológiai területek és kihívások

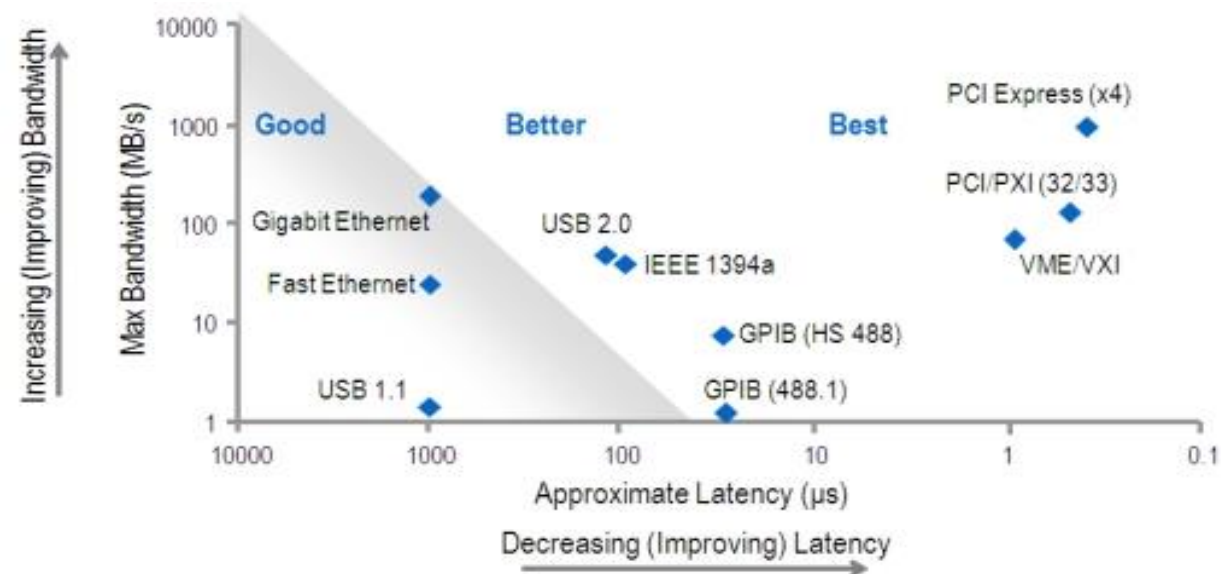
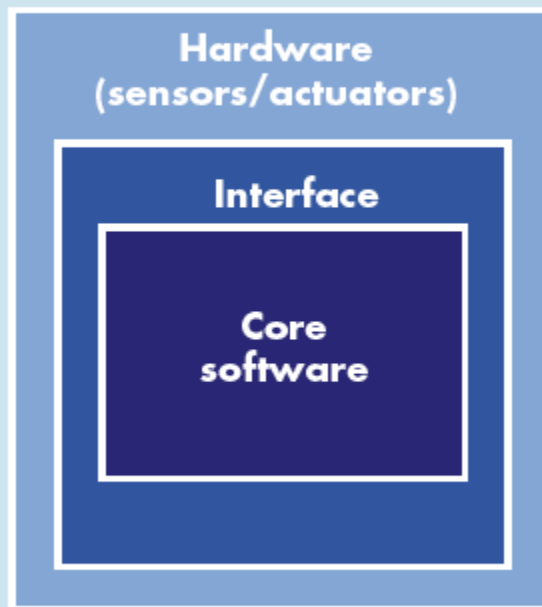
---

- 5. Mindenütt jelenlevő vezeték nélküli kommunikáció:**  
korlátozott sáv szélesség, tápáram-igény, protokollok, ...
- 6. „Szilícium-skálázás”:** bizonyítottan helyes működésű IP blokkok hatékony újrafelhasználásának módszerei, ...
- 7. Érzékelők és beavatkozók:** érzékelő hálózatok, MEMS-ek (Micro-Electro-Mechanical Systems), rádió frekvencia identifikáció (RFID), ...
- 8. Ember-gép interfészek:** a tudással kapcsolatos (kognitív) modelleket és a felhasználó viselkedését megragadó módszerek kutatása, ...
- 9. Elosztott számítási platform:** elosztott számítási eszközök, kooperativitás és adaptivitás az eszközök működésében, ...
- 10. Önszervező, adaptív rendszerek:** kooperativitás és adaptivitás a környezet megváltozásának hatására, ...

# Felépítés



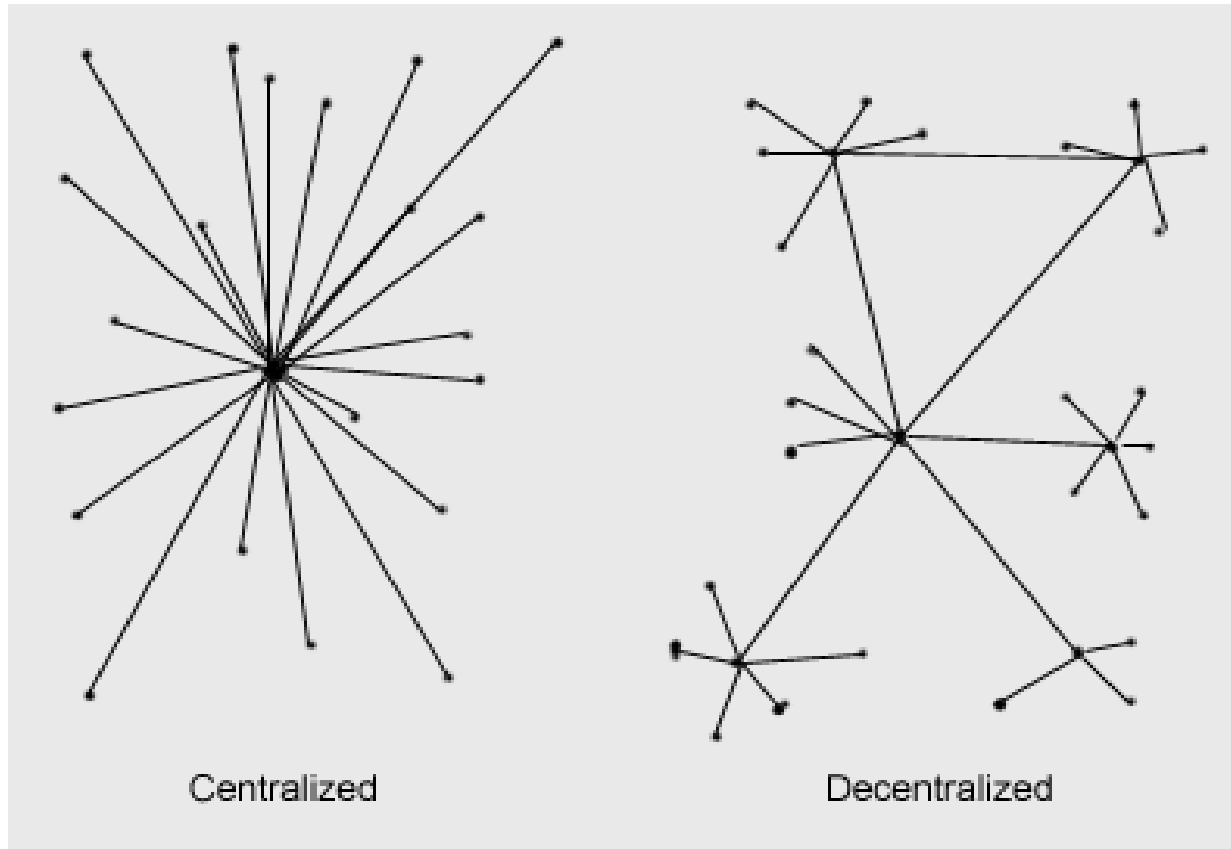
## Perceived system behavior



# Beágyazott rendszer hálózatok

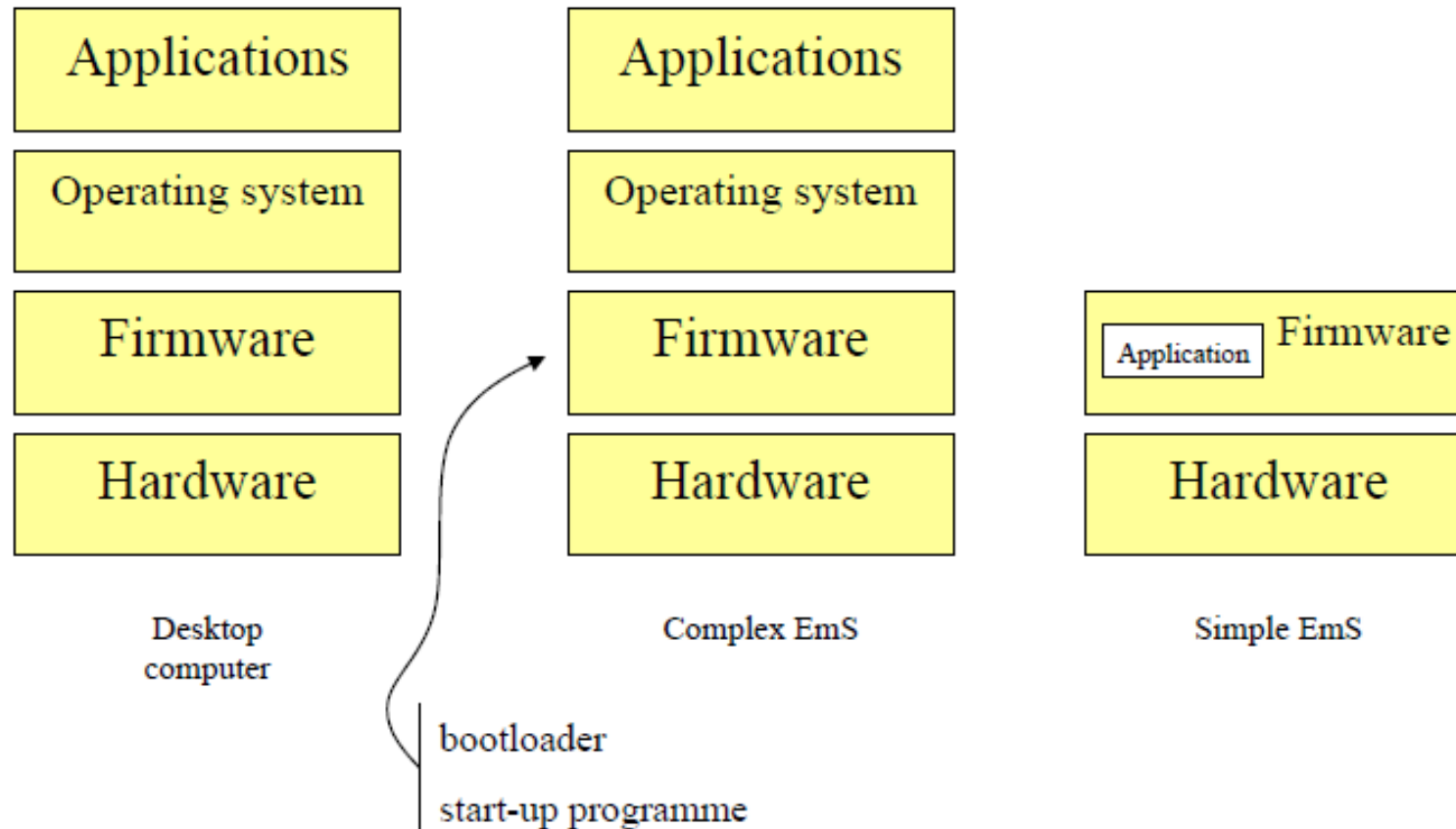
---

Általában elosztott vezérlőrendszerre épülnek



# Software szintek

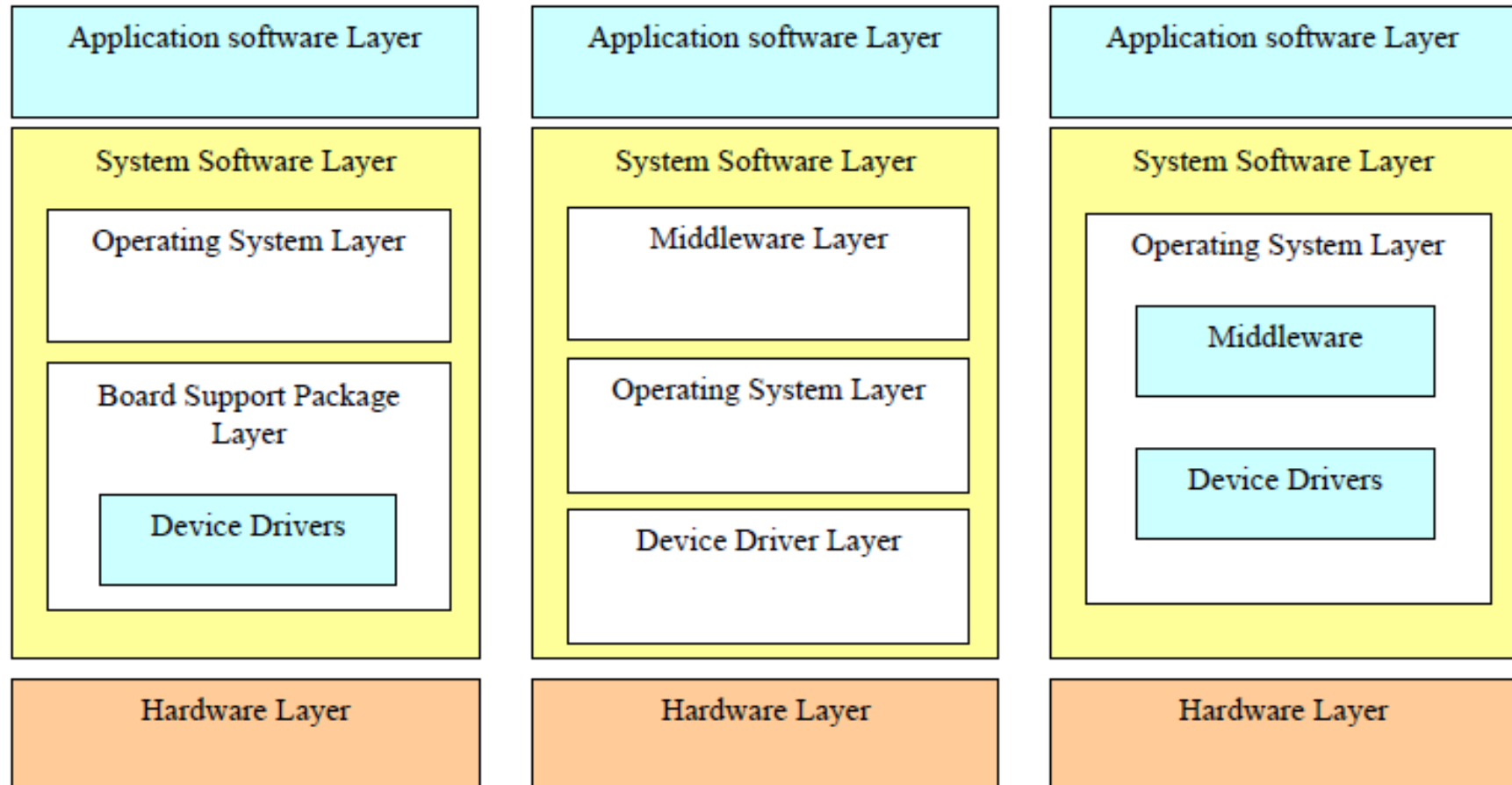
---



**EmS – Embedded System**

# EMS operációs rendszerek

---



# EmS tervezésének fázisai

---

## 1. Fázis: Architektúra tervezése

- A termék követelményeinek meghatározása
- Funkcionális specifikációk kidolgozása
- Architektúra projekt létrehozása
- Architektúra terv véglegesítése

## 2. Fázis: Architektúra kiépítése

- Alegységek (modulok) implementálása
- Ellenőrzés (szimuláció) és hibajavítása
- A rendszer kiépítése és integrálása

## 3. Fázis: Rendszer tesztelése

- A rendszer ellenőrzése és tesztelése az implementáció különböző fázisaiban

## 4. Fázis: Prototípus gyártása

# Párhuzamosítási módszerek

---

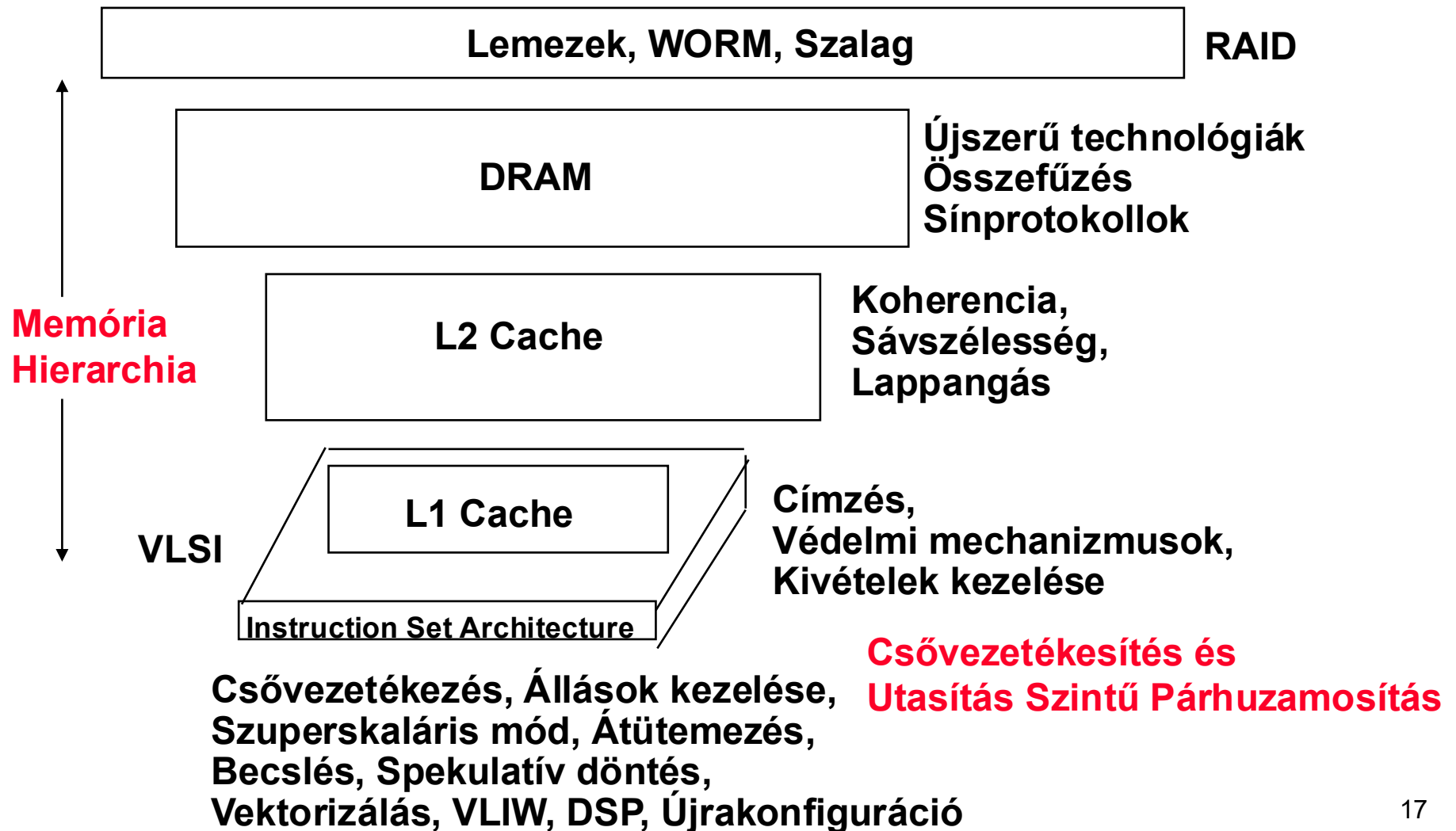
- **PC-k CPU-ja már nem képzelhetők el ezek nélkül**
  - **Perifériás eszközökben is jelen vannak (grafikus proc.)**
  - **Az EmS rendszerekben is fokozottan jelentkező igény**
- 
- **Tekintsük át a legfontosabb stratégiákat**

***Beágyazott rendszerek  
teljesítmény-, költség- és energiahatékonysági  
analízise***

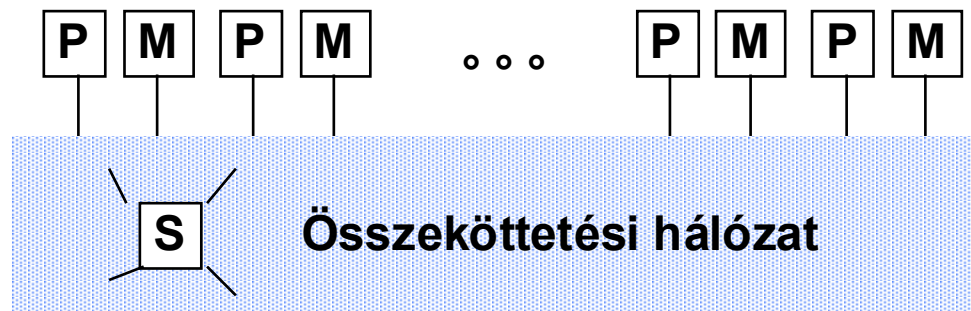


# Architektúra témakörök

## Ki/Bementek és Tárolók



# Tervezési szempontok



Processzor-Memória-Switch

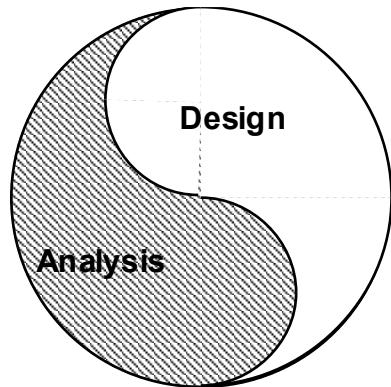
**Multiprocesszorok**  
**Hálózat és csatlakoztatás**

**Osztott Memória,  
Üzenetközvetítés,  
Adatpárhuzamosság**

**Hálózati illesztők**

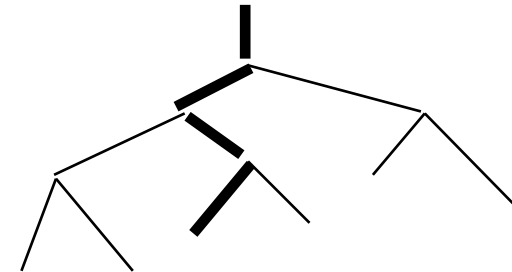
**Topológiák,  
Routing,  
Sávszélesség,  
Lappangási idők,  
Megbízhatóság**

# Mérés és kiértékelés



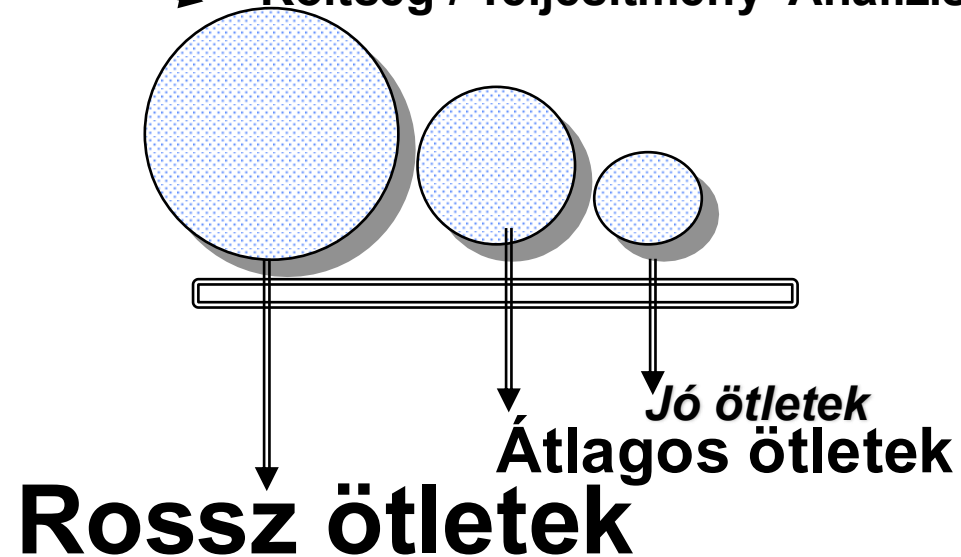
## A tervezés egy iteratív folyamat:

- Keresés a lehetséges tervek terében
- A beágyazott rendszerek minden szintjének elemzése

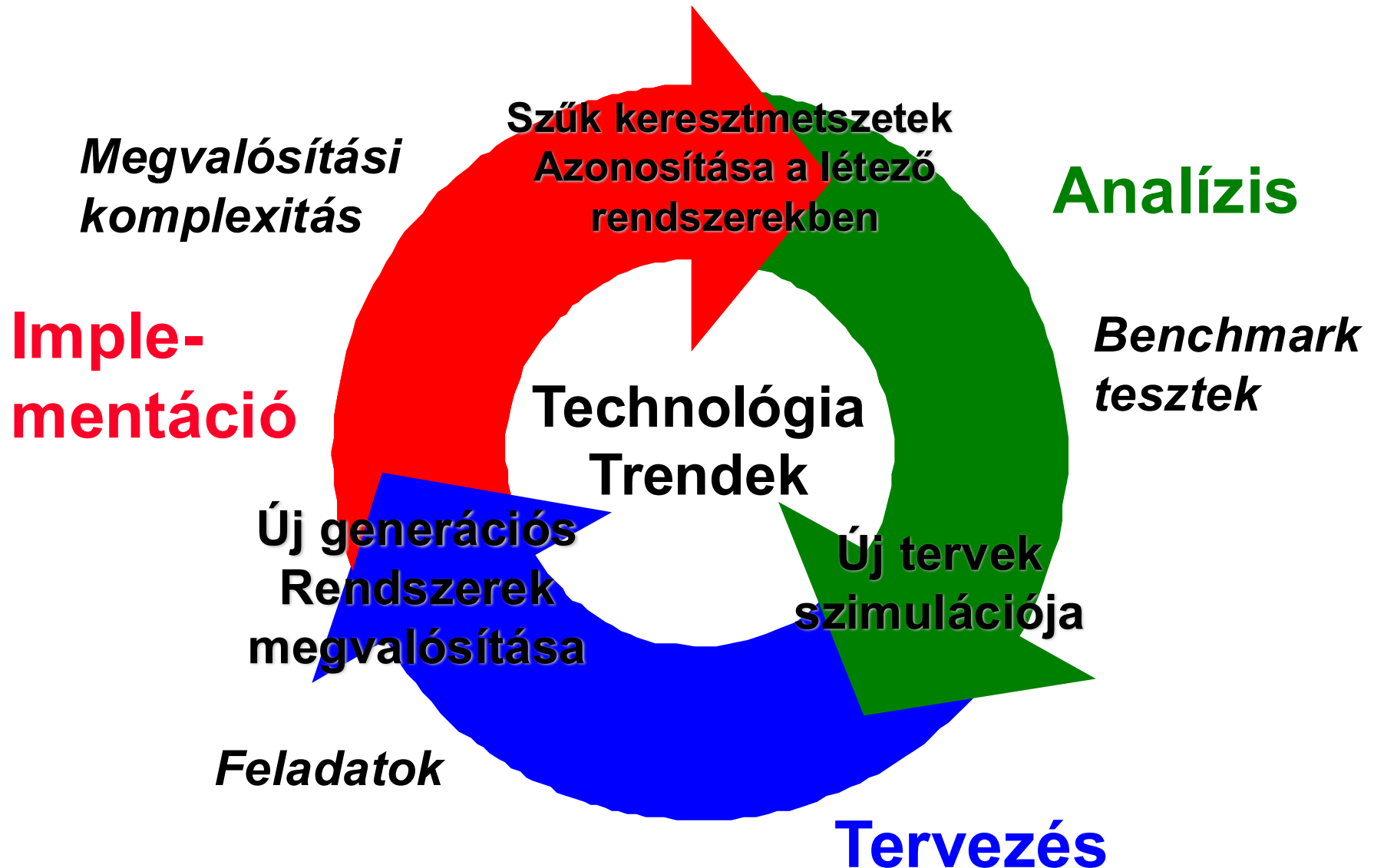


**Kreativitás**

**Költség / Teljesítmény Analízis**



# *Tervezési módszertan*



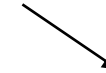
# Mérési eszközök

- **Hardware: Költség, késleltetés, erőforrások, teljesítmény becslés**
- **Benchmark tesztek, Trace-ek (végrehatás követés)**
- **Szimuláció (sok szintű)**
  - ISA, RTL, Kapu, Áramkör
- **Ütemezési elmélet (Queuing)**
- **Rules of Thumb (ököl szabály, azaz egy gyakorlati elv, amely nem minden esetben pontos, de könnyen megjegyezhető - heurisztika)**
- **Alapvető “Törvények”/Elvek**

***Teljesítmény, költség, energia***

# 1. Metrika : Teljesítmény

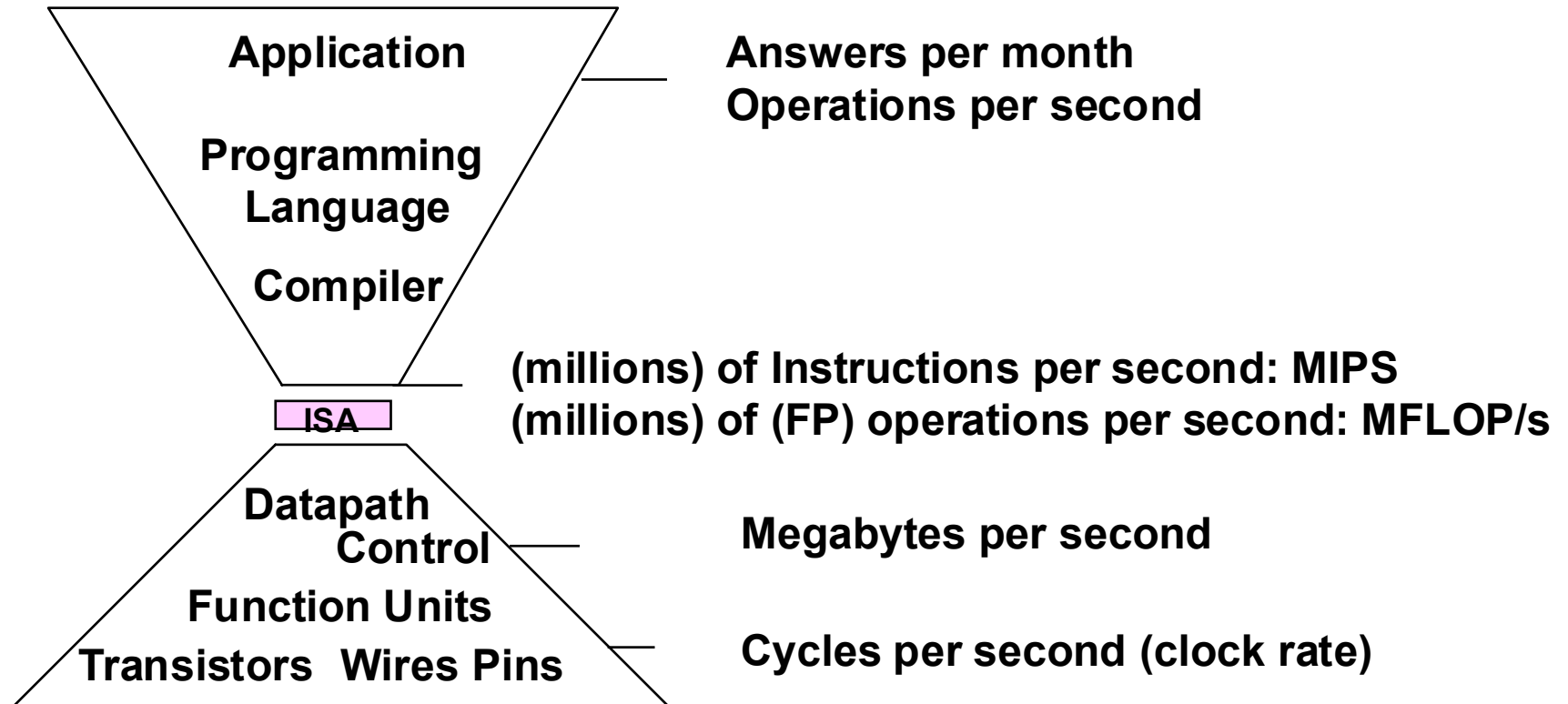
In passenger-mile/hour



| Plane      | DC to Paris | Speed    | Passengers | Throughput |
|------------|-------------|----------|------------|------------|
| Boeing 747 | 6.5 hours   | 610 mph  | 470        | 286,700    |
| Concorde   | 3 hours     | 1350 mph | 132        | 178,200    |

- **Time to run the task**
  - Execution time, response time, latency
- **Tasks per day, hour, week, sec, ns ...**
  - Throughput, bandwidth

# Teljesítmény metrikák





# CPU teljesítmény szempontok

$$\text{CPU time} = \frac{\text{Seconds}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Cycle}}$$

|                     | Inst Count | CPI        | Clock Rate |
|---------------------|------------|------------|------------|
| <b>Program</b>      | <b>X</b>   |            |            |
| <b>Compiler</b>     | <b>X</b>   | <b>(X)</b> |            |
| <b>Inst. Set.</b>   | <b>X</b>   | <b>X</b>   |            |
| <b>Organization</b> |            | <b>X</b>   | <b>X</b>   |
| <b>Technology</b>   |            |            | <b>X</b>   |

# Cycles Per Instruction

## “Average Cycles per Instruction”

$$\begin{aligned}\text{CPI} &= \text{Cycles} / \text{Instruction Count} \\ &= (\text{CPU Time} * \text{Clock Rate}) / \text{Instruction Count}\end{aligned}$$

$$\text{CPU time} = \text{CycleTime} * \sum_{i=1}^n \text{CPI}_i * I_i$$

## “Instruction Frequency”

$$\text{CPI} = \sum_{i=1}^n \text{CPI}_i * F_i \quad \text{where } F_i = \frac{I_i}{\text{Instruction Count}}$$

**Invest Resources where time is Spent!**

## *Példa: CPI (clocks/instr.) számítás*

**Base Machine (Reg / Reg)**

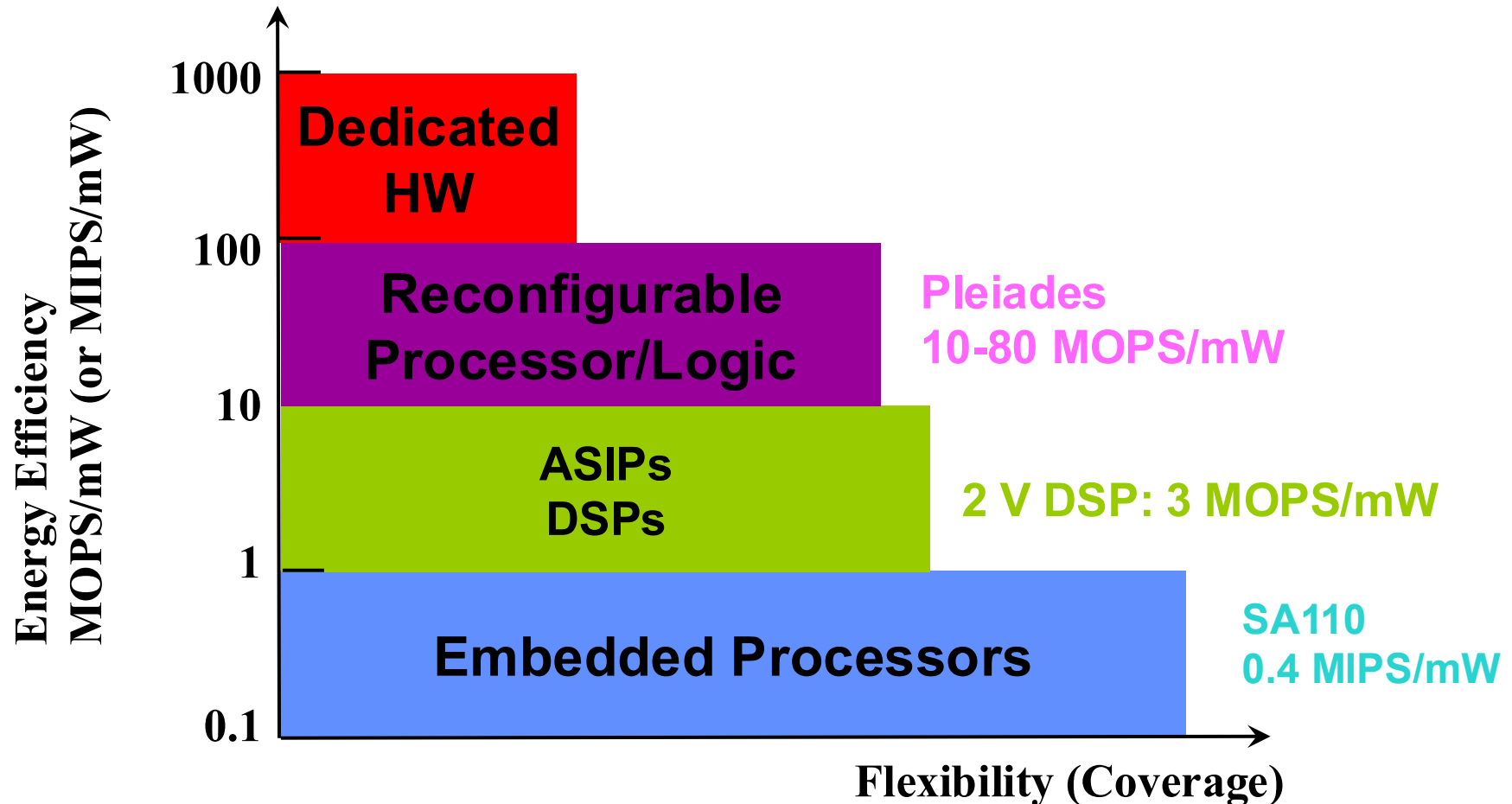
| Op     | Freq | CPI <sub>i</sub> | CPI <sub>i</sub> *F <sub>i</sub> | (% Time) |
|--------|------|------------------|----------------------------------|----------|
| ALU    | 50%  | 1                | .5                               | (33%)    |
| Load   | 20%  | 2                | .4                               | (27%)    |
| Store  | 10%  | 2                | .2                               | (13%)    |
| Branch | 20%  | 2                | .4                               | (27%)    |
|        |      |                  | <hr/>                            |          |
|        |      |                  | 1.5                              |          |

Typical Mix

# Hogyan foglaljuk össze a teljesítményt?

- **Arithmetic mean** (súlyozott számtani átlag) követi a végrehajtási időt:  $\Sigma(T_i)/n$  or  $\Sigma(W_i * T_i)$
- **Harmonic mean** (súlyozott harmonikus átlag) az arányokhoz (pl. MFLOPS) követi a végrehajtási időt:  $n / \Sigma(1/R_i)$  or  $n / \Sigma(W_i / R_i)$
- **Normalized execution time** hasznos a teljesítmény skálázására (pl. X-szer gyorsabb, mint a SPARCstation 10)
  - Az aritmetikai átlagot befolyásolja a referencia gép választása
- Használjunk **geometriai átlagot** az összehasonlításához:  $\prod(T_i)^{1/n}$ 
  - Független a választott géptől
  - De nem jó mérőszám a teljes végrehajtási időre

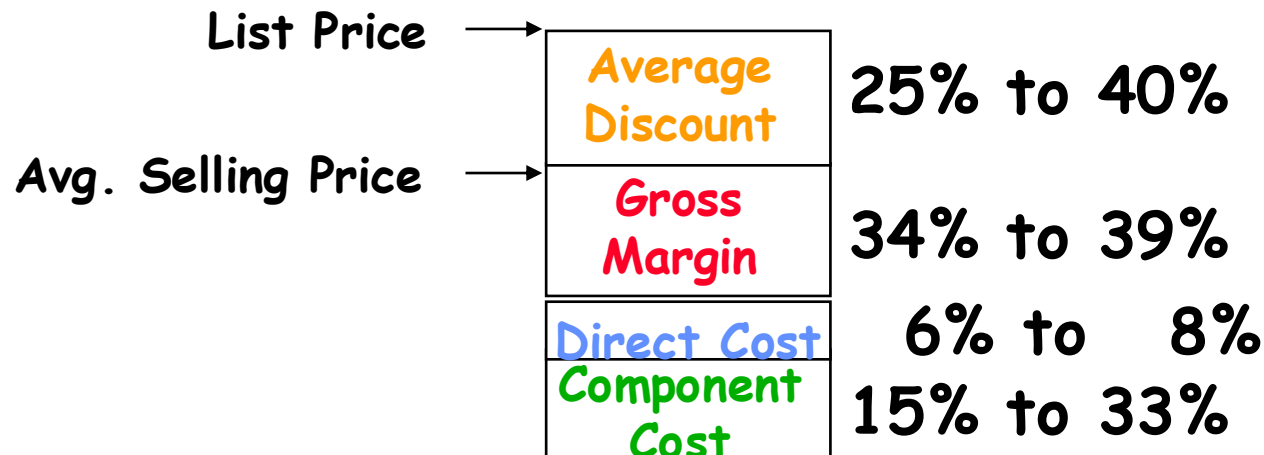
# *The Energy-Flexibility Gap – Energia-képlékenység összefüggés*



# Cost/Performance – Költség/Teljesítmény

*Mi a kapcsolat a költség és az ár között?*

- **Ismétlődő költségek**
  - **Komponens költségek**
  - **Közvetlen költségek** ((25-40%-kal növelve), például munkaerő, beszerzés, selejt, garancia)
- **Nem ismétlődő költségek vagy bruttó árrés** (82-186%-kal növelt)  
(K+F, berendezés-karbantartás, bérlet, marketing, értékesítés, finanszírozási költség, adózás előtti nyereség, adók)
- **Átlagos kedvezmény** a listaár eléréséhez (33-66%-kal növelt):  
menyiségi kedvezmények és/vagy kiskereskedői árrés



# Összefoglaló

- Amdahl törvénye:

$$\text{Speedup}_{\text{overall}} = \frac{\text{ExTime}_{\text{old}}}{\text{ExTime}_{\text{new}}} = \frac{1}{(1 - \text{Fraction}_{\text{enhanced}}) + \frac{\text{Fraction}_{\text{enhanced}}}{\text{Speedup}_{\text{enhanced}}}}$$

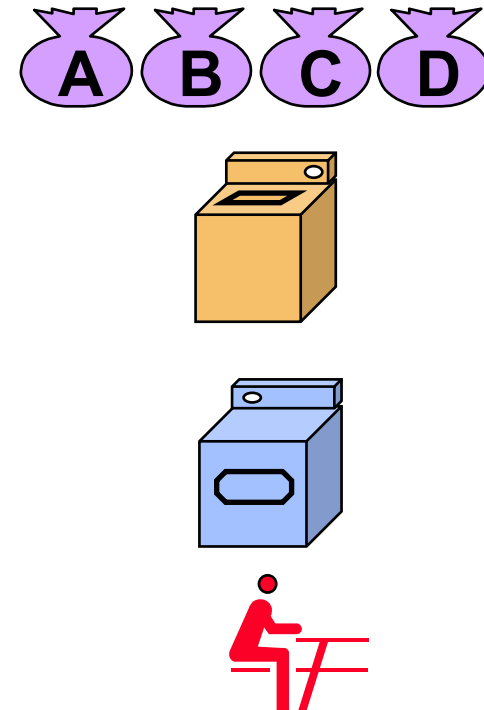
- A CPI törvény:

|          |   |                                         |   |                                              |   |                                            |   |                                       |
|----------|---|-----------------------------------------|---|----------------------------------------------|---|--------------------------------------------|---|---------------------------------------|
| CPU time | = | $\frac{\text{Seconds}}{\text{Program}}$ | = | $\frac{\text{Instructions}}{\text{Program}}$ | x | $\frac{\text{Cycles}}{\text{Instruction}}$ | x | $\frac{\text{Seconds}}{\text{Cycle}}$ |
|----------|---|-----------------------------------------|---|----------------------------------------------|---|--------------------------------------------|---|---------------------------------------|

- **A végrehajtási idő a számítógépes teljesítmény VALÓDI mércéje!**
- Jó termékek akkor születnek, ha van:
  - Jó benchmarkok, jó módszerek a teljesítmény összegzésére
- **Beágyazott rendszerek esetén eltérő metrikák alkalmazandók.**

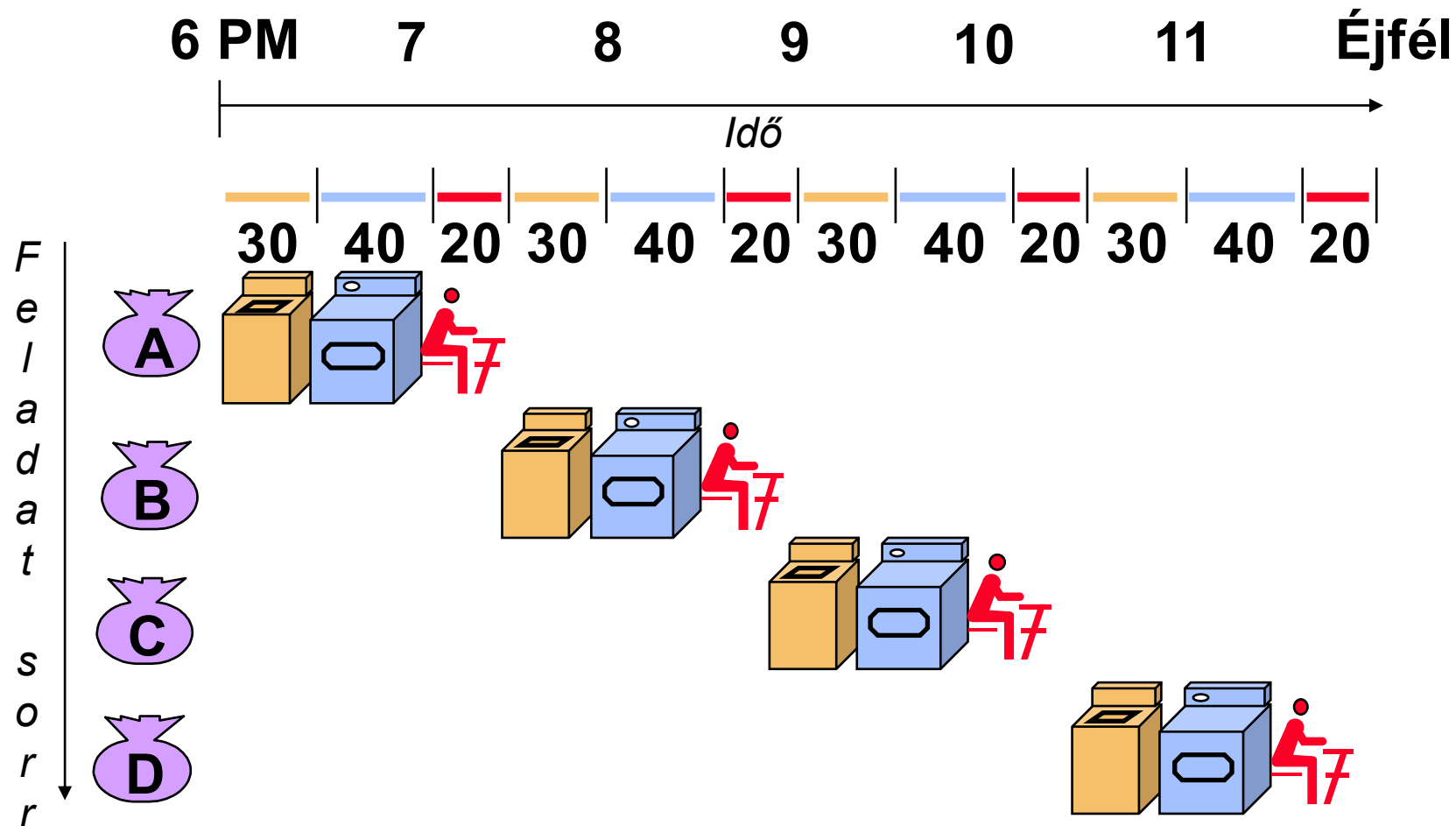
# Csővezetékesítés.... szemléltető példa

- Mosási példa
- Ann-nek, Brian-nek, Cathy-nek és Dave-nek van egy-egy adag ruhája, amit mosni, szárítani és hajtogatni kell.
- A mosógép 30 percet vesz igénybe.
- A szárítógép 40 percet vesz igénybe
- A „hajtogató” 20 percet vesz igénybe.



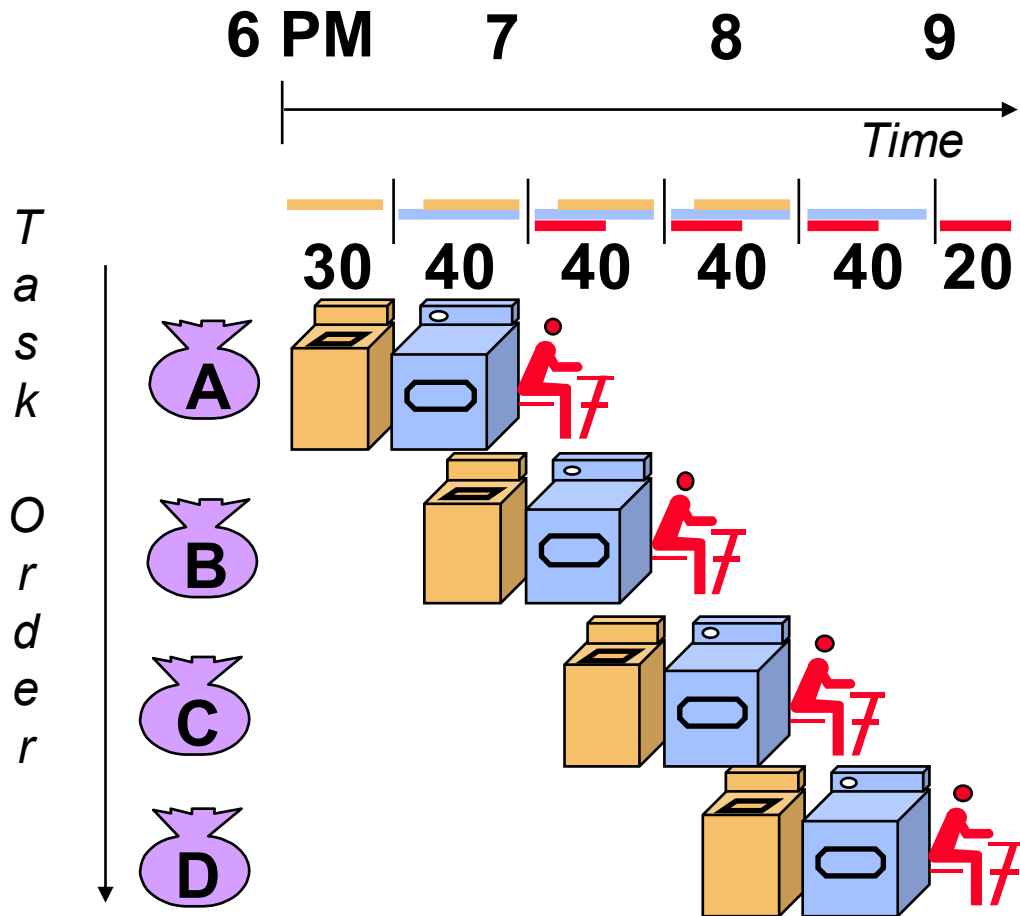


# Soros mosás



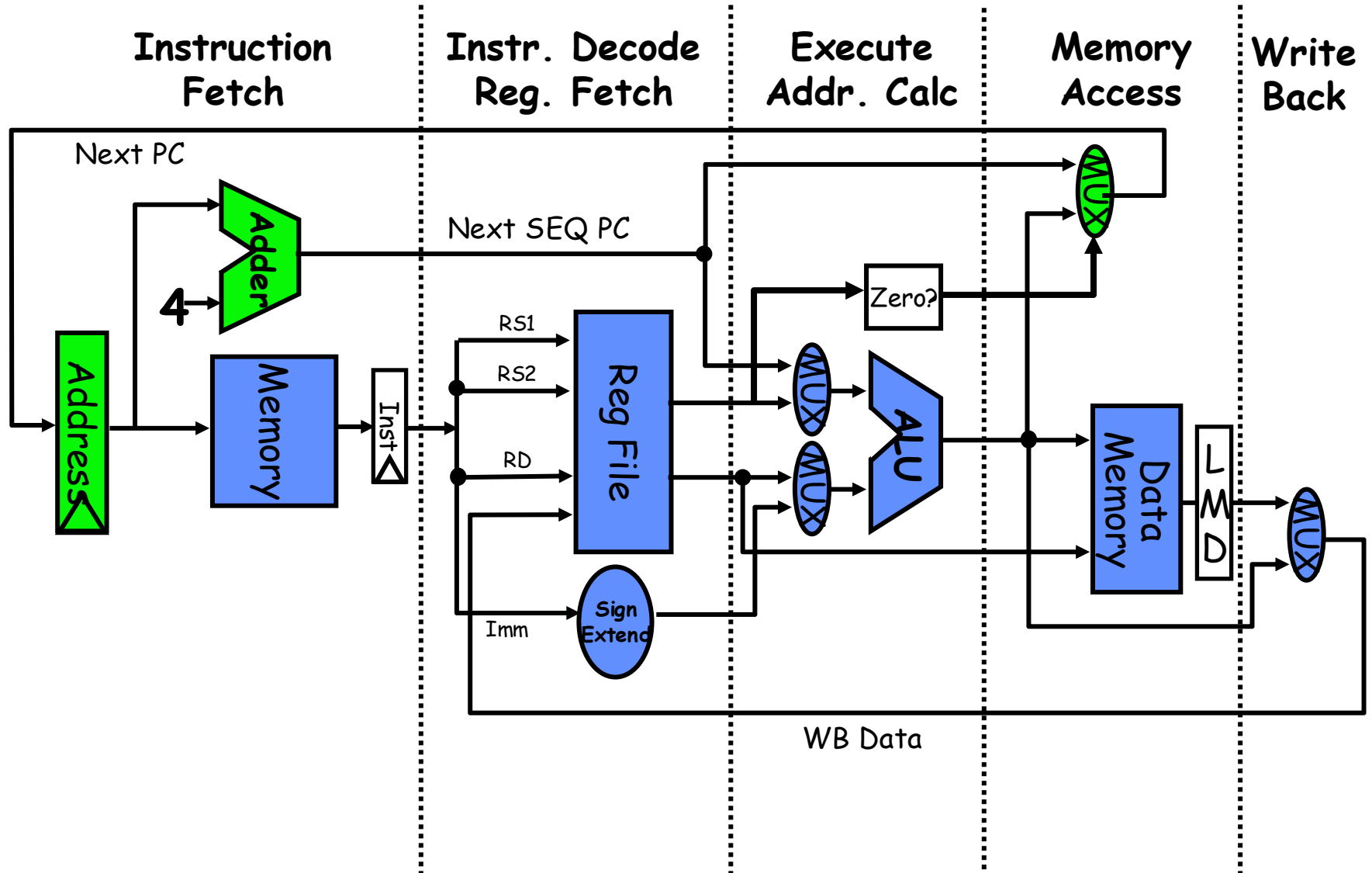
- A soros mosás 6 órát vesz igénybe 4 adag ruhához.
- Ha megtanulnák a csővezetékes (pipelining) módszert, mennyi ideig tartana a mosás?

# Csővezetékes feldolgozás tanulságai

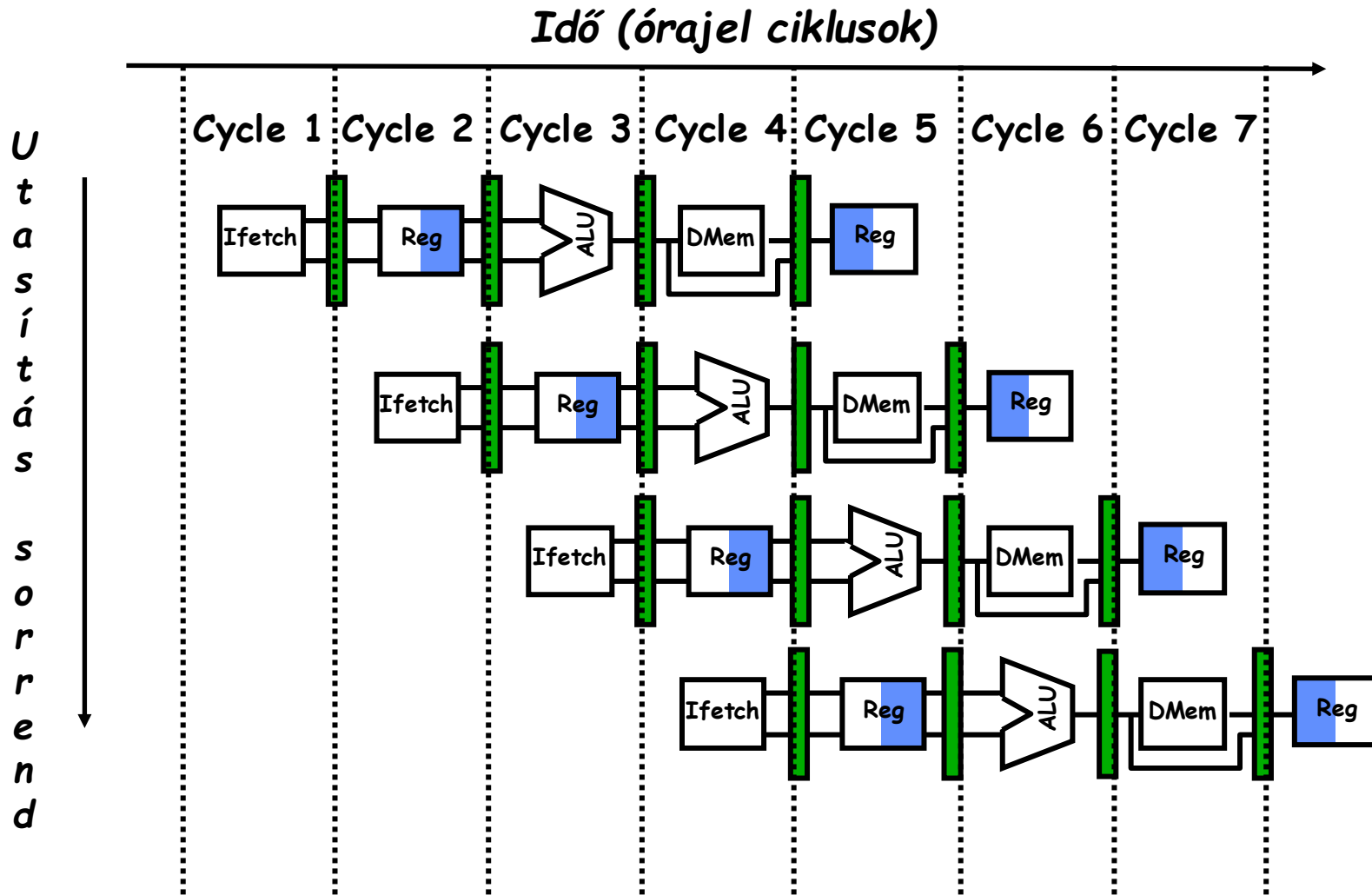


- A csővezetékes feldolgozás nem egyetlen feladat **késleltetését** csökkenti, hanem az egész munkaterhelés **áteresztő-képességét** növeli.
- A csővezeték sebességét a **leglassabb** szakasz korlátozza.
- **Több** feladat működik egyidejűleg.
- Lehetséges gyorsulás = **csővezeték-szakaszok száma**.
- Az egyensúlyban nem lévő csővezeték-szakaszok hossza csökkenti a gyorsulást.
- A csővezeték „**feltöltési**” és „**leeresztési**” ideje csökkenti a gyorsulást.

# A csővezeték adatút 5 lépése



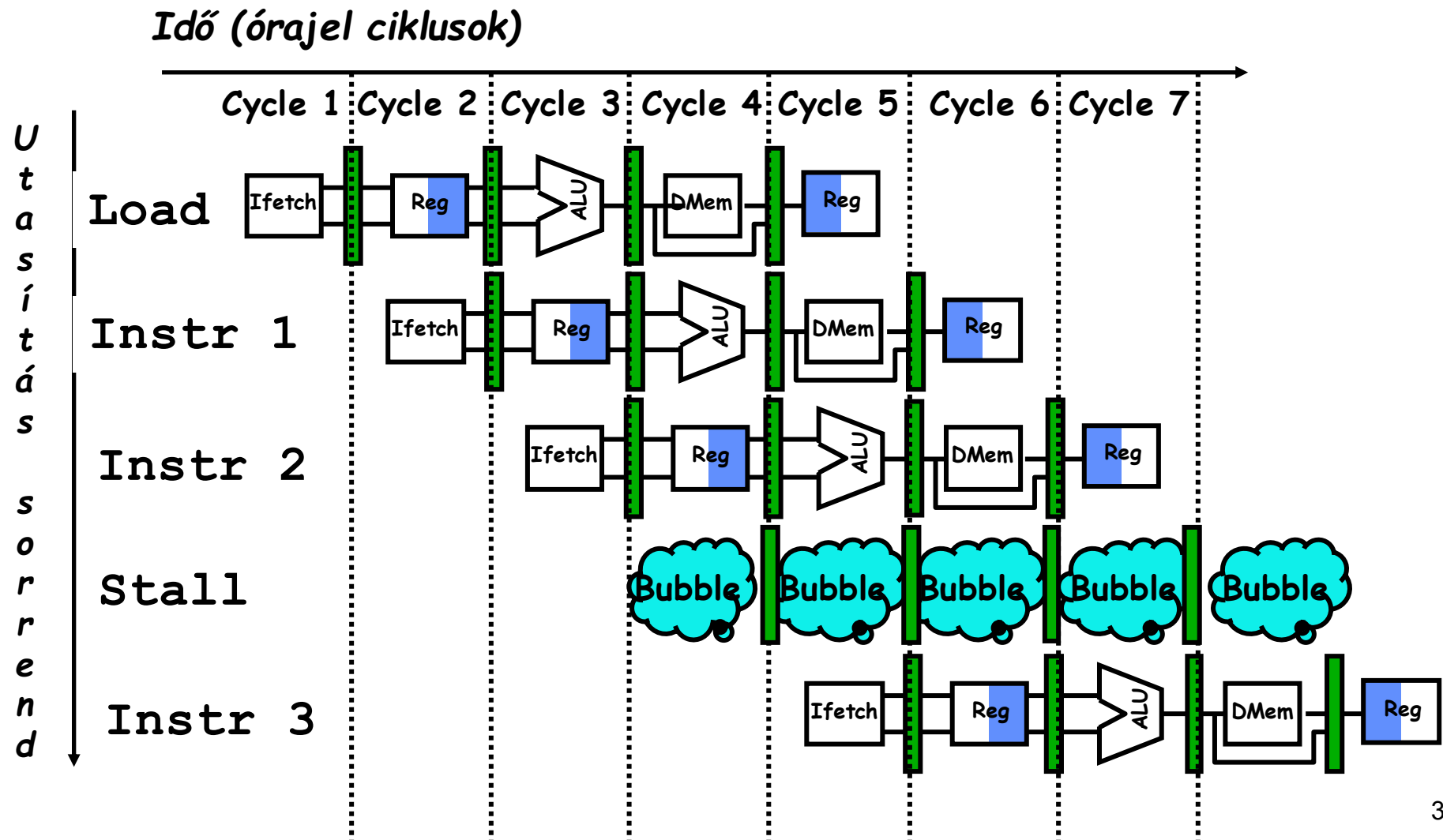
# A csővezeték vizualizálása



# Nem olyan egyszerű a processzorok számára

- A csővezetékes feldolgozás korlátai:  
Az **ütközések** megakadályozzák a következő utasítás végrehajtását a kijelölt órajelciklus alatt.
  - **Strukturális ütközések**: a hardver nem képes támogatni ezt az utasításkombinációt – két kutya küzd ugyanazért a csontért.
  - **Adatütközések**: az utasítás a korábbi utasítás eredményére támaszkodik, amely még mindig a csővezetékben van.
  - **Vezérlési ütközések**: az utasítások betöltése és a vezérlési áramlás változtatásainak döntései (elágazások és ugrások) közötti késedelem okozza

# Egy memória port / Strukturális ütközések



# A csővezetékes gyorsulás egyenlete

$$CPI_{\text{pipelined}} = \text{Ideal CPI} + \text{Average Stall cycles per Inst}$$

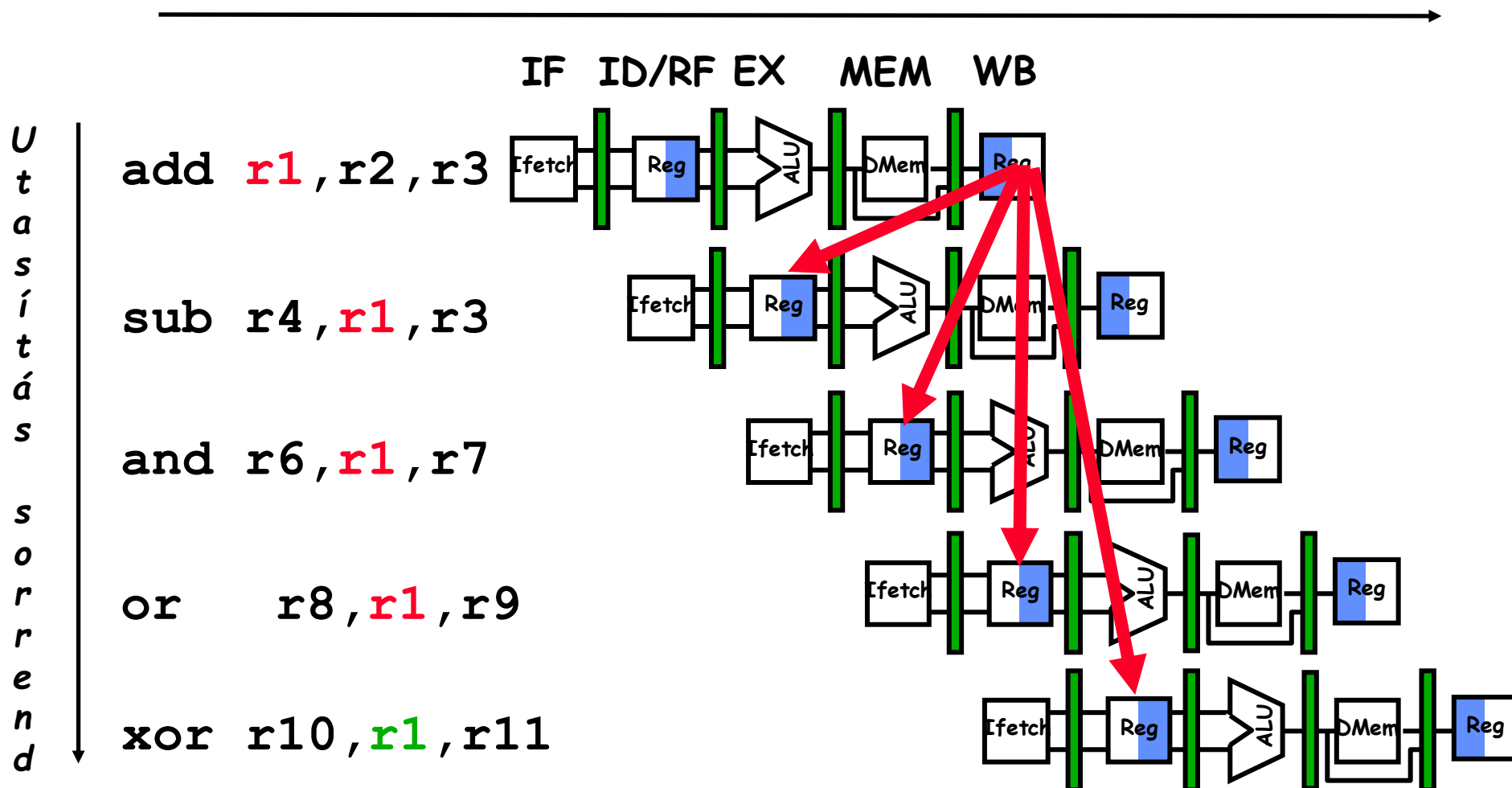
$$\text{Speedup} = \frac{\text{Ideal CPI} \times \text{Pipeline depth}}{\text{Ideal CPI} + \text{Pipeline stall CPI}} \times \frac{\text{Cycle Time}_{\text{unpipelined}}}{\text{Cycle Time}_{\text{pipelined}}}$$

Egyszerű RISC csővezeték esetén,  $CPI = 1$ :

$$\text{Speedup} = \frac{\text{Pipeline depth}}{1 + \text{Pipeline stall CPI}} \times \frac{\text{Cycle Time}_{\text{unpipelined}}}{\text{Cycle Time}_{\text{pipelined}}}$$

# Adatütközés (Data Hazard) az R1-en

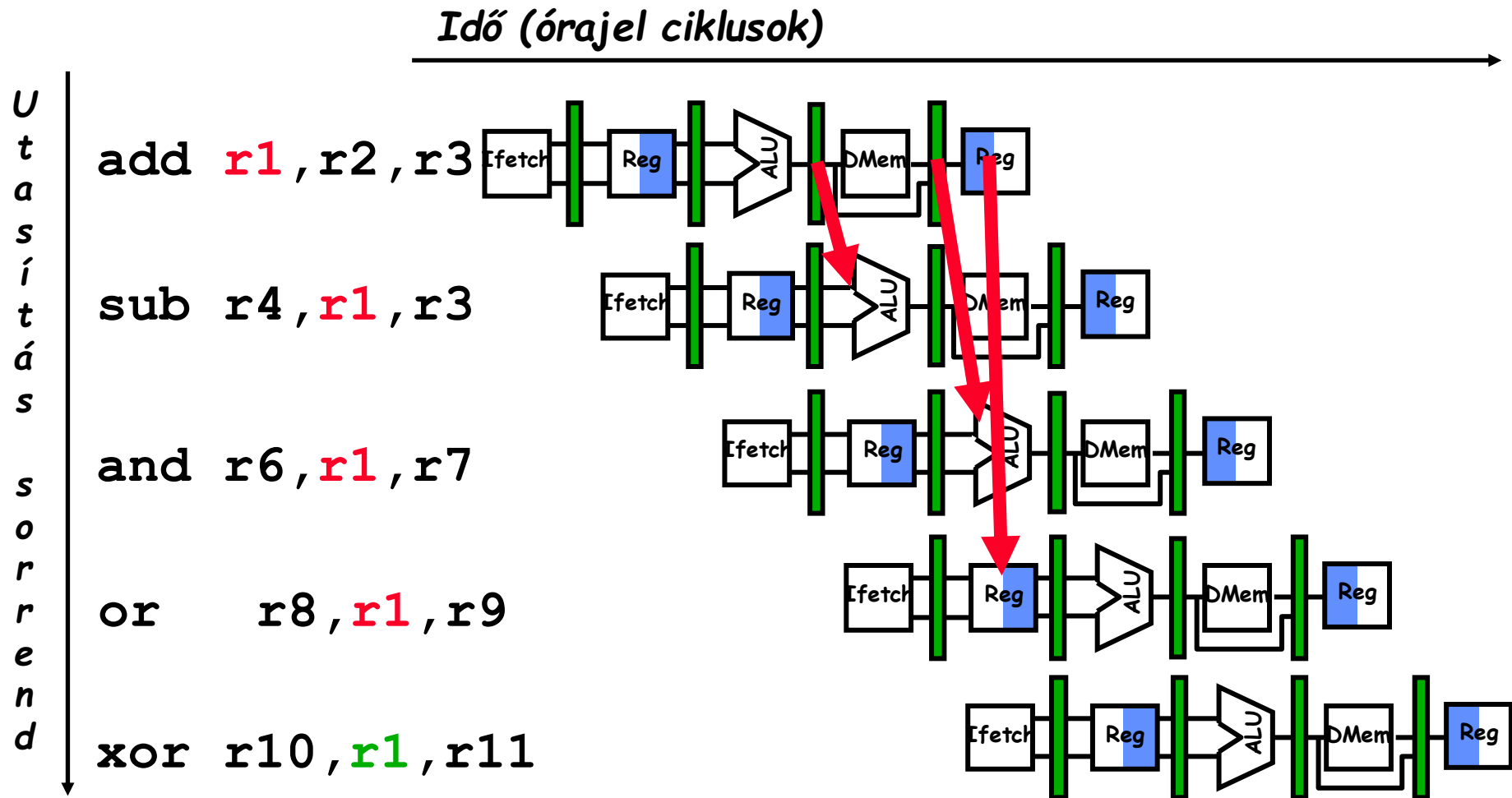
Idő (órajel ciklusok)





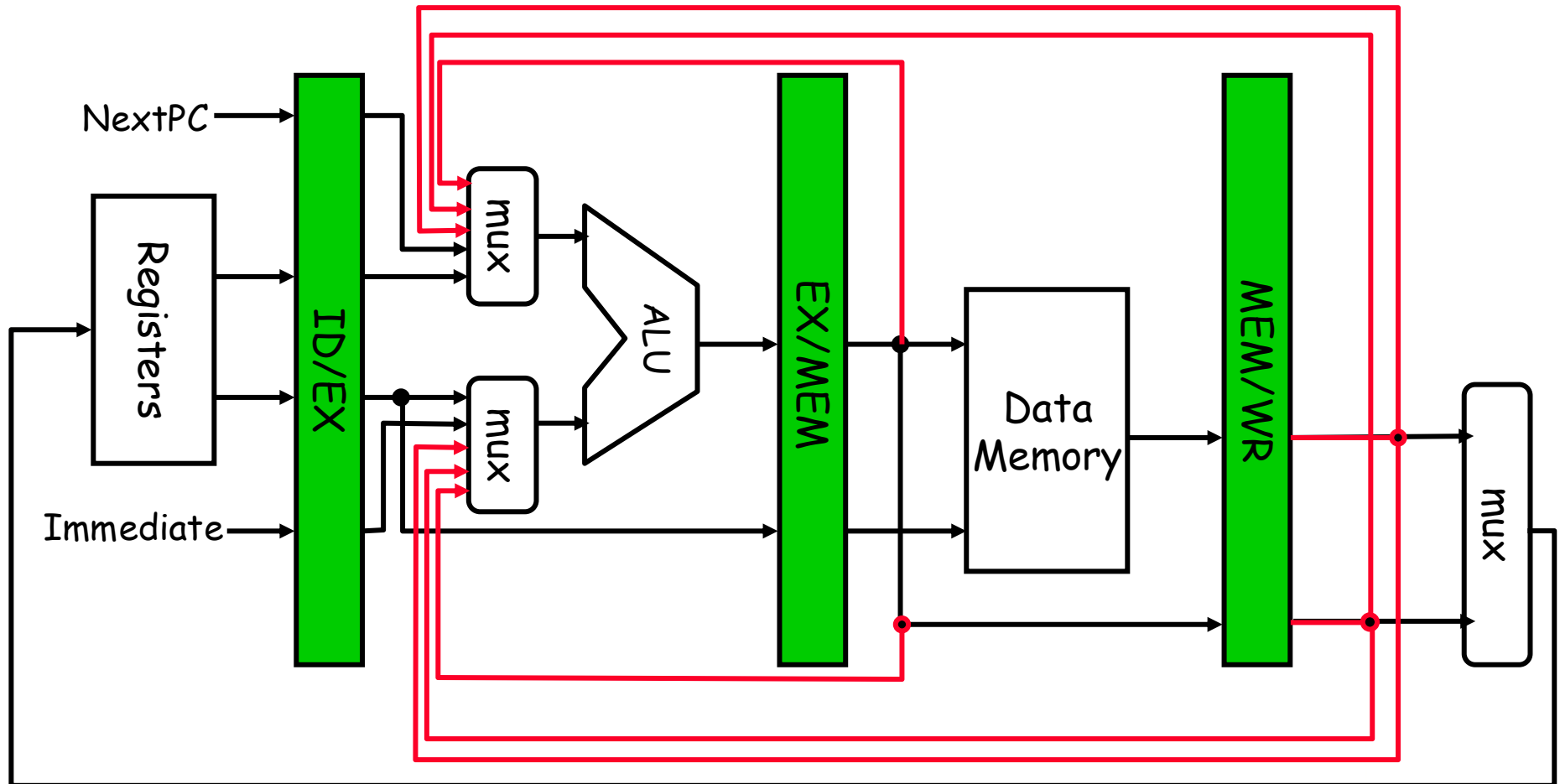
# Előrehozott értékátvitel az adatvesztély elkerülésére

## Forwarding to Avoid Data Hazard



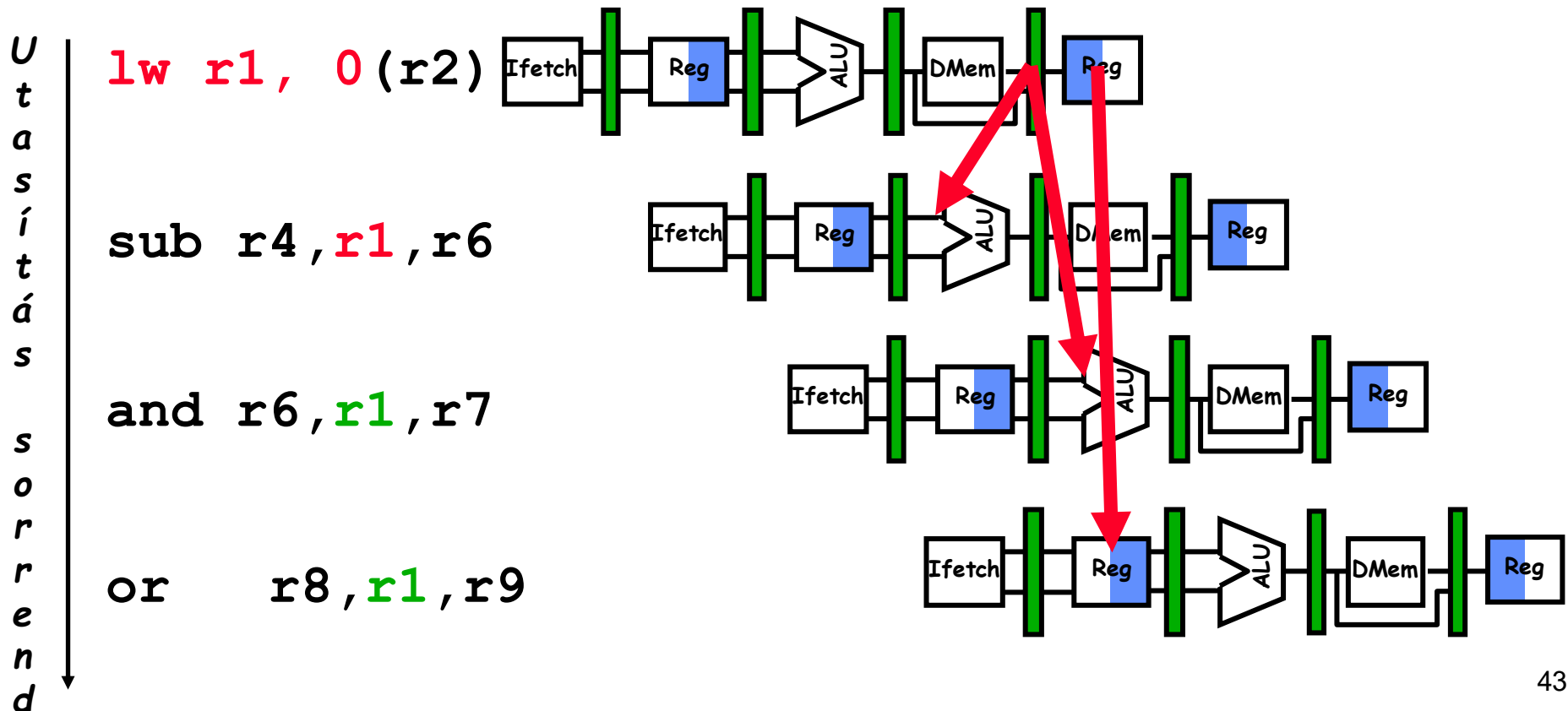
# Hardvermódosítás az előretoábbításhoz

## HW Change for Forwarding



# Adatveszély előreátvitel mellett is Data Hazard Even with Forwarding

Idő (órajel ciklusok)



# *Adatveszély előretovábbítás mellett is Data Hazard Even with Forwarding*

◆ **Előretovábbítás (Forwarding)** segít, de nem old meg minden adatveszélyt.

**Mikor fordulhat elő mégis?**

**1 Memória-betöltési késleltetés (Load-Use Hazard)**

- Az adat még nem érhető el a memóriából.

Példa:

```
LW  R1, 0(R2)    ; Memória olvasás  
ADD R3, R1, R4    ; R1 még nem kész!
```

**2 Több lépéses végrehajtás (Multi-cycle Execution)**

- Pl. lebegőpontos műveletek, szorzás/osztás.

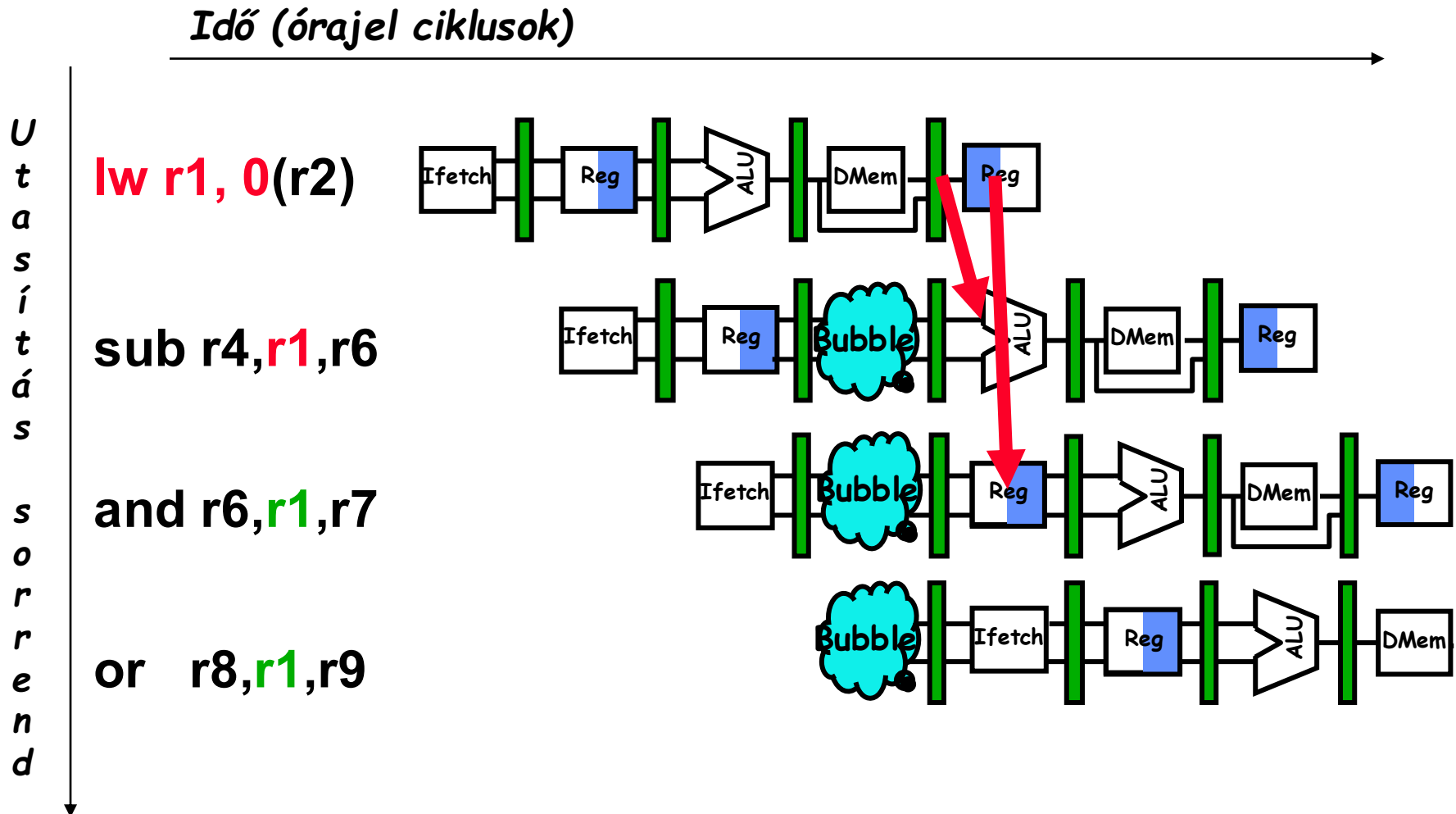
**3 Előretovábbítási konfliktusok**

- Több utasítás egyszerre próbálja használni az adatot.

**Megoldás:**

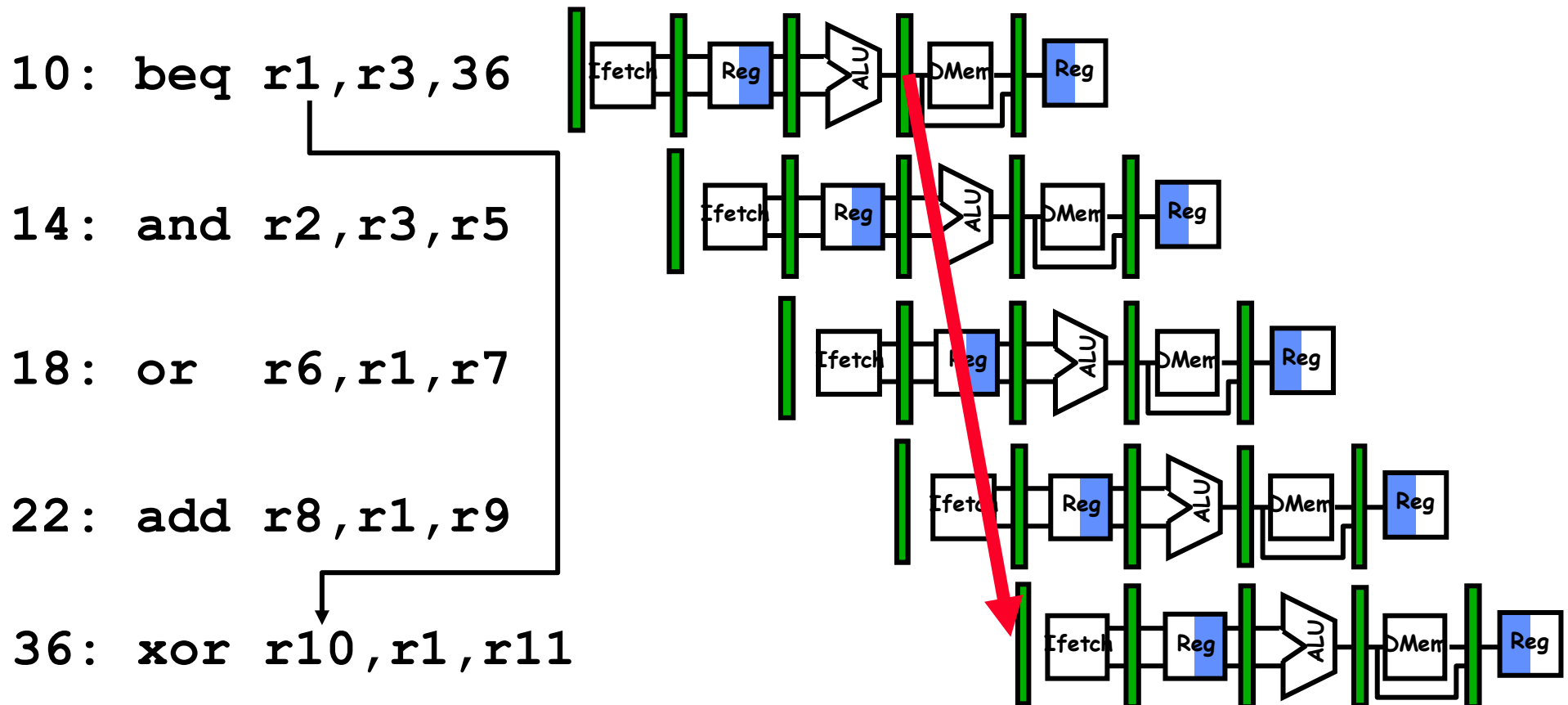
- ✓ **Pipeline késleltetés (Stalling)** – üres ciklusok beszúrása.

# Adatveszély előretoábbítás mellett is Data Hazard Even with Forwarding



# Vezérlési veszély elágazásoknál & Háromlépcsős késleltetés

## Control Hazard on Branches - Three Stage Stall



# *Vezérlési veszély elágazásoknál & Háromlépcsős késleltetés*

## *Control Hazard on Branches - Three Stage Stall*

◆ **Vezérlési veszély (Control Hazard)** akkor fordul elő, amikor a processzor nem tudja előre, melyik utasítást kell végrehajtani egy elágazás (branch) után.

**Megoldás: Háromlépcsős késleltetés (Three Stage Stall)**

1. **Elágazás felismerése** (Branch Decode)
2. **Célcím kiszámítása** (Target Address Calculation)
3. **Döntés végrehajtásáról** (Branch Execution)

✗ **Hátrány:** A csővezeték három ciklusra megállhat.

✓ **Megoldások:** Előzetes elágazásbecslés (Branch Prediction) vagy Késleltetett ugrás (Delayed Branching).

# *Elágazási késleltetés hatása*

- Ha  $CPI = 1$ , 30%-os elágazás,  
Késleltetés: 3 ciklus  $\Rightarrow$  új  $CPI = 1,9!$
- Két részből álló megoldás:
  - határozd meg, hogy az elágazást végrehajtották-e hamarabb, ÉS
  - számítsd ki az elágazott címeket korábban.
- A DLX elágazás ellenőrzi, hogy a regiszter = 0 vagy  $\neq 0$ .
- DLX megoldás:
  - mozgasd a Nulla tesztet az ID/RF szakaszba
  - összeadó az új PC kiszámításához az ID/RF szakaszban
  - 1 órajelciklus büntetés az elágazásért szemben a 3-al.

A DLX (ejtsd "Deluxe") egy RISC processzor architektúra, amelyet John L. Hennessy (Stanford MIPS) és A. Patterson (Berkeley RISC) tervezett.



# Ágazati alternatívák értékelése

$$\text{Pipeline speedup} = \frac{\text{Pipeline depth}}{1 + \text{Branch frequency} \times \text{Branch penalty}}$$

| <i>Scheduling scheme</i> | <i>Branch penalty</i> | <i>CPI</i> | <i>speedup v. unpipelined</i> | <i>speedup v. stall</i> |
|--------------------------|-----------------------|------------|-------------------------------|-------------------------|
| Stall pipeline           | 3                     | 1.42       | 3.5                           | 1.0                     |
| Predict taken            | 1                     | 1.14       | 4.4                           | 1.26                    |
| Predict not taken        | 1                     | 1.09       | 4.5                           | 1.29                    |
| Delayed branch           | 0.5                   | 1.07       | 4.6                           | 1.31                    |

**Conditional & Unconditional = 14%, 65% change PC**

# Összefoglaló 2

- Egyszerűen átfedjük a feladatokat; könnyű, ha a feladatok függetlenek.
- Gyorsulás  $\leq$  Csővezeték mélysége; ha az ideális CPI 1, akkor:

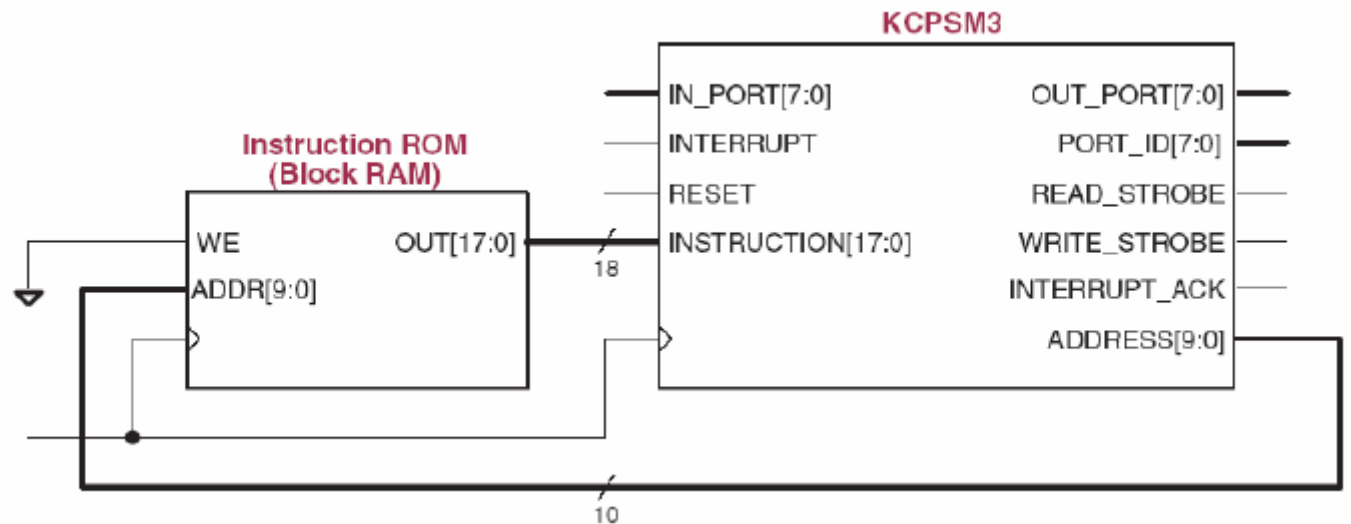
$$\text{Speedup} = \frac{\text{Pipeline depth}}{1 + \text{Pipeline stall CPI}} \times \frac{\text{Cycle Time}_{\text{unpipelined}}}{\text{Cycle Time}_{\text{pipelined}}}$$

- Az ütközések korlátozzák a számítógépek teljesítményét:
  - Strukturális: további hardver erőforrásokra van szükség
  - Adat (RAW, WAR, WAW): átirányításra, fordítói ütemezésre van szükség
  - Vezérlési: késlekedett elágazás, előrejelzés

# Multi-core rendszer FPGA platformon

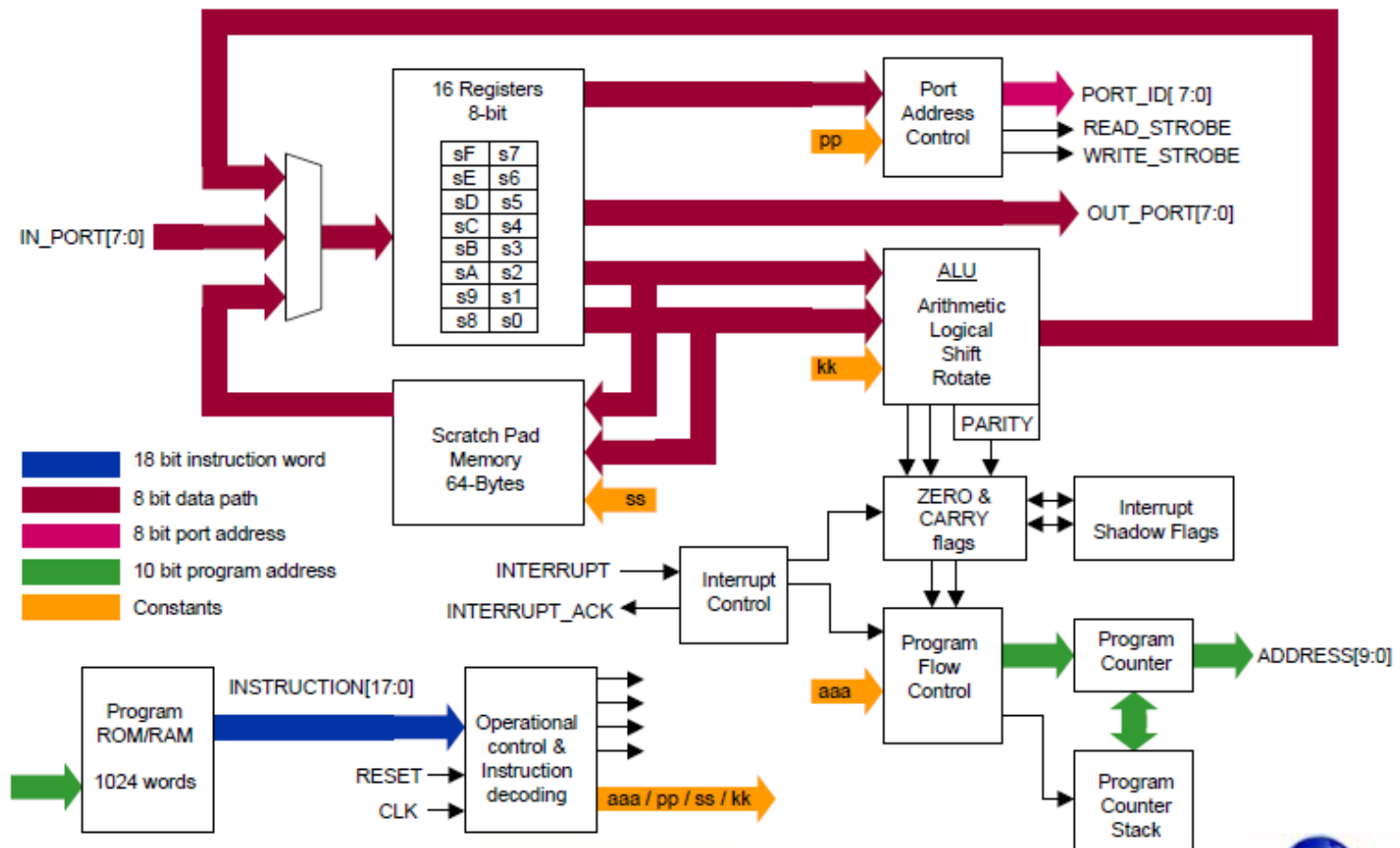
- ❖ 8 bit RISC
- ❖ Többszörösen példányosítható
- ❖ kis erőforrás igényű
- ❖ max. 87MHz
- ❖ 2 Clock cycles / instruction
- ❖ 96 slices

**PicoBlaze™**



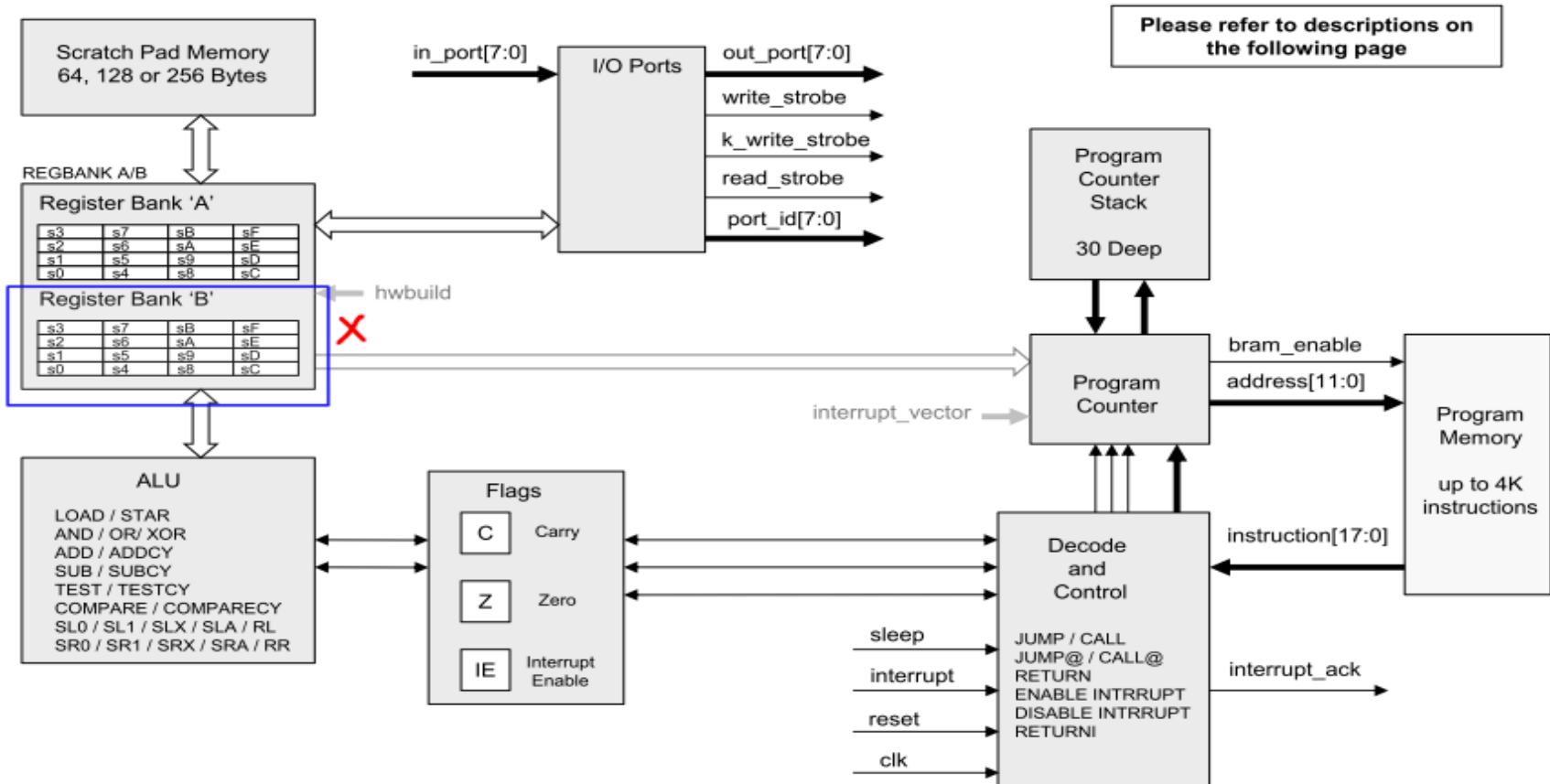
# PicoBlaze architektúra

## KCPSM3 Architecture



# PicoBlaze architektúra

## KCPSM6 Architecture and Features

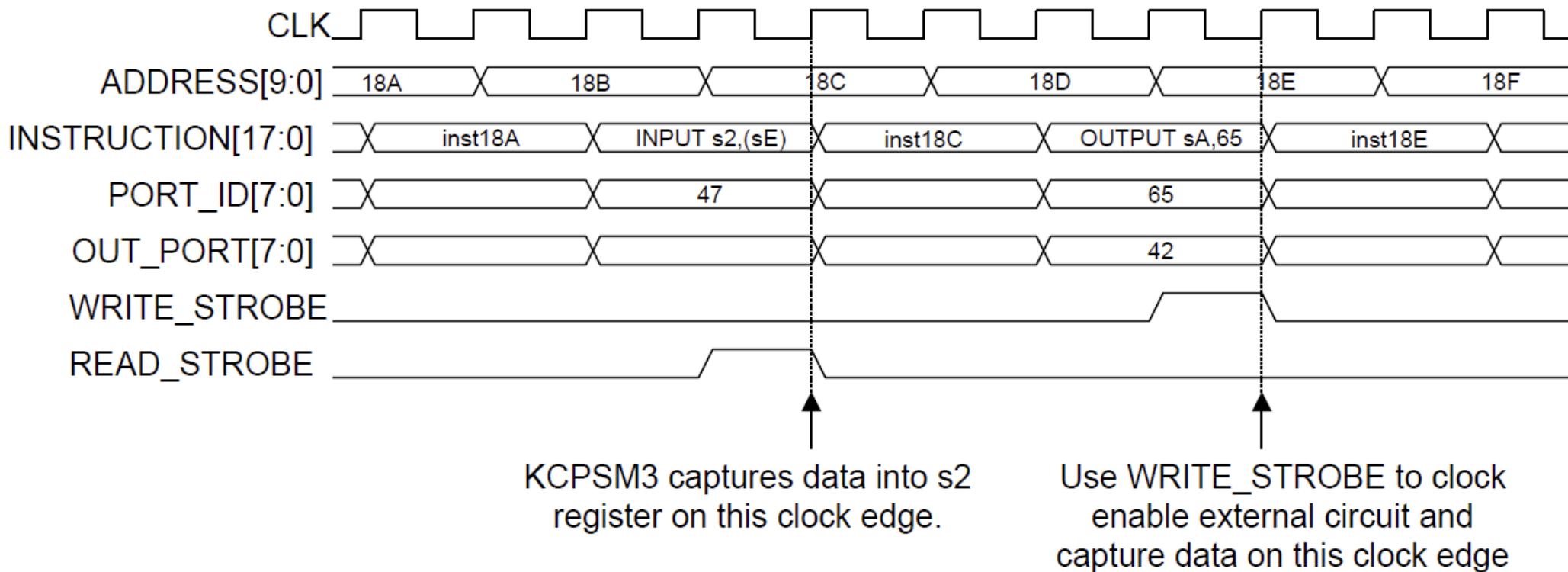


Page 6

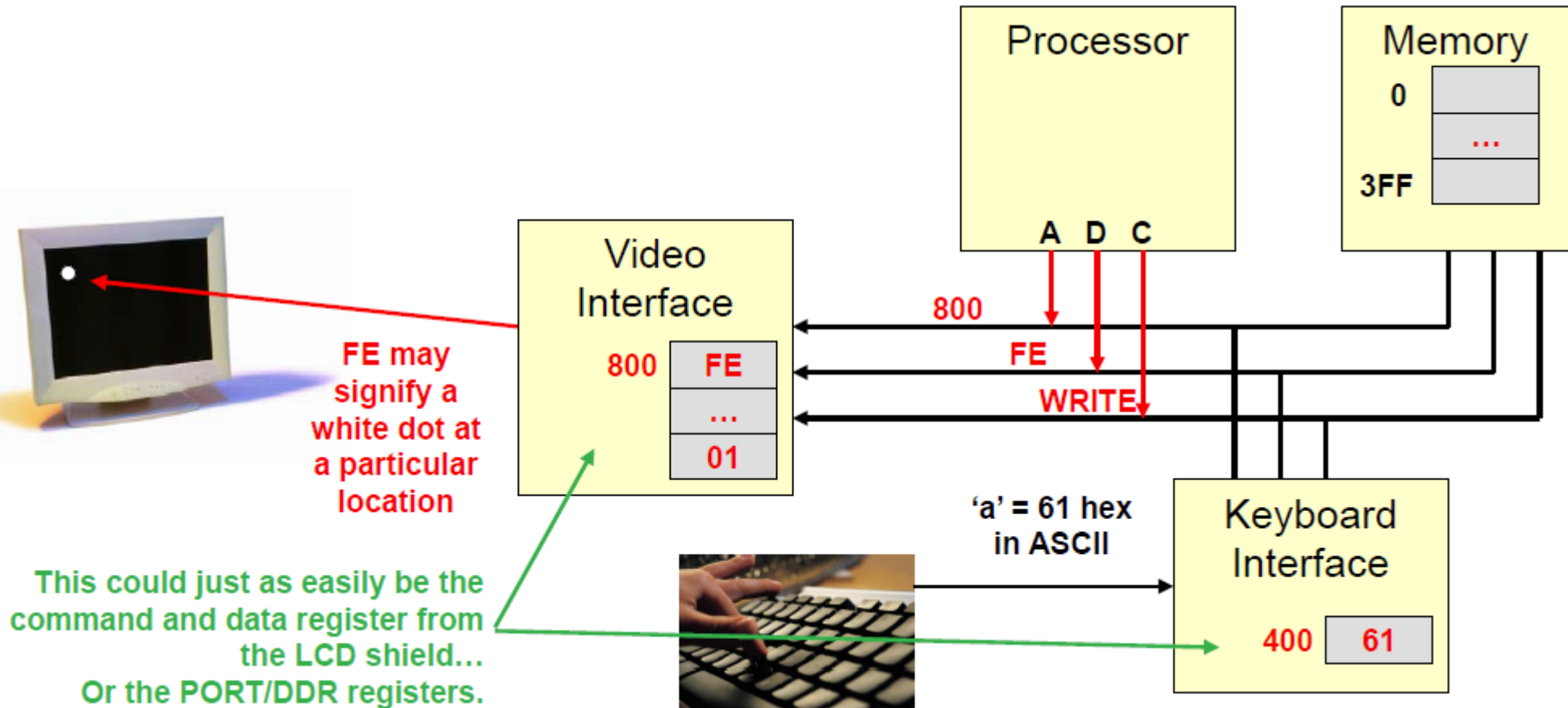
© Copyright 2010-2014 Xilinx

XILINX

# Port-műveletek

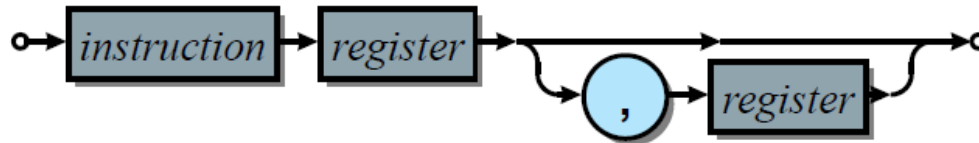


# PicoBlaze SoC – Példa



# PicoBlaze SoC – Utasítás készlet

## Register



## Immediate



|               |                                                                                  |
|---------------|----------------------------------------------------------------------------------|
| <instruction> | Instruction mnemonic or directive                                                |
| <register>    | One of the 16 currently active registers (s0-sF)                                 |
| <literal>     | Constant literal (hex [nn], decimal [nnn'd], binary [nnnnnnnn'b], or char ["c"]) |

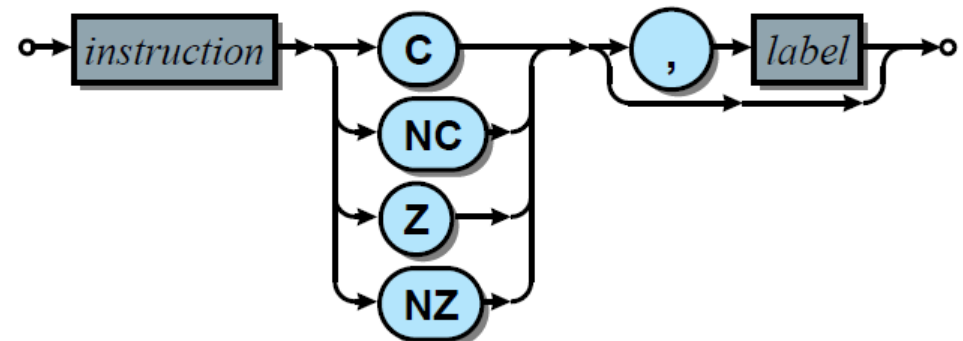
## Indirect scratchpad / I/O port



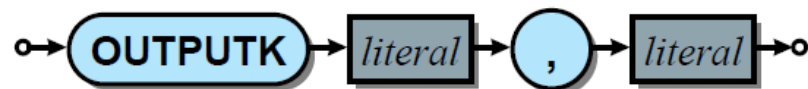
## Indirect jump / call



## Conditional



## Outputk





# PicoBlaze SoC – Utasítás készlet

## KCPSM6 Instruction Set

aaa : 12-bit address 000 to FFF  
 kk : 8-bit constant 00 to FF  
 pp : 8-bit port ID 00 to FF  
 p : 4-bit port ID 0 to F  
 ss : 8-bit scratch pad location 00 to FF  
 x : Register within bank s0 to sF  
 y : Register within bank s0 to sF

Page Opcode Instruction

### Register loading

55 00xy0 LOAD sX, sY  
 55 01xkk LOAD sX, kk  
 71 16xy0 STAR sX, sY  
 71 17xkk STAR sX, kk

### Logical

56 02xy0 AND sX, sY  
 56 03xkk AND sX, kk  
 57 04xy0 OR sX, sY  
 57 05xkk OR sX, kk  
 58 06xy0 XOR sX, sY  
 58 07xkk XOR sX, kk

### Arithmetic

59 10xy0 ADD sX, sY  
 59 11xkk ADD sX, kk  
 60 12xy0 ADDCY sX, sY  
 60 13xkk ADDCY sX, kk  
 61 18xy0 SUB sX, sY  
 61 19xkk SUB sX, kk  
 62 1Axy0 SUBCY sX, sY  
 62 1Bxkk SUBCY sX, kk

### Test and Compare

63 0Cxy0 TEST sX, sY  
 63 0Dxkk TEST sX, kk  
 64 0Exy0 TESTCY sX, sY  
 64 0Fxkk TESTCY sX, kk  
 65 1Cxy0 COMPARE sX, sY  
 65 1Dxkk COMPARE sX, kk  
 66 1Exy0 COMPARECY sX, sY  
 66 1Fxkk COMPARECY sX, kk

Page 54

Page Opcode Instruction

### Shift and Rotate

67 14x06 SLO sX  
 67 14x07 SLI sX  
 67 14x04 SLX sX  
 67 14x00 SLA sX  
 67 14x02 RL sX  
 68 14x0E SRO sX  
 68 14x0F SRI sX  
 68 14x0A SRX sX  
 68 14x08 SRA sX  
 68 14x0C RR sX

### Register Bank Selection

70 37000 REGBANK A  
 70 37001 REGBANK B

### Input and Output

73 08xy0 INPUT sX, (sY)  
 73 09xpp INPUT sX, pp  
 74 2Cxy0 OUTPUT sX, (sY)  
 74 2Dxpp OUTPUT sX, pp  
 78 2Bkkp OUTPUTK kk, p

### Scratch Pad Memory

(64, 128 or 256 bytes)

81 2Exy0 STORE sX, (sY)  
 81 2Fxss STORE sX, ss  
 82 0Axy0 FETCH sX, (sY)  
 82 0Bxss FETCH sX, ss

Page Opcode Instruction

### Interrupt Handling

83 28000 DISABLE INTERRUPT  
 83 28001 ENABLE INTERRUPT  
 84 29000 RETURNI DISABLE  
 84 29001 RETURNI ENABLE

### Jump

87 22aaa JUMP aaa  
 88 32aaa JUMP Z, aaa  
 88 36aaa JUMP NZ, aaa  
 88 3Aaaa JUMP C, aaa  
 88 3Eaaa JUMP NC, aaa  
 89 26xy0 JUMPO (sX, sY)

### Subroutines

92 20aaa CALL aaa  
 93 30aaa CALL Z, aaa  
 93 34aaa CALL NZ, aaa  
 93 38aaa CALL C, aaa  
 93 3Caaa CALL NC, aaa  
 94 24xy0 CALLO (sX, sY)  
 96 25000 RETURN  
 97 31000 RETURN Z  
 97 35000 RETURN NZ  
 97 39000 RETURN C  
 97 3D000 RETURN NC

98 21xkk LOAD&RETURN sX, kk

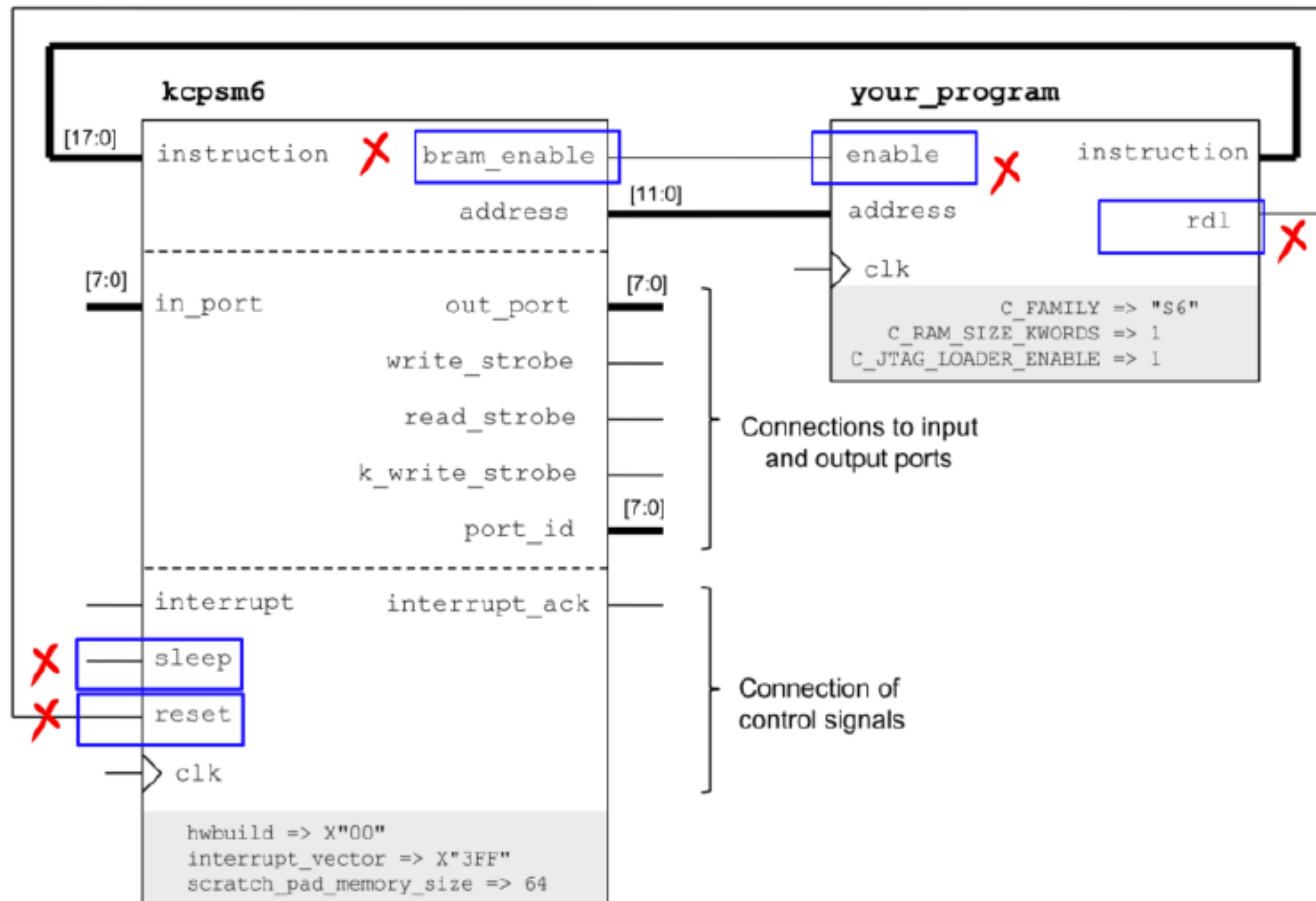
### Version Control

101 14x80 HWBUILD sX

© Copyright 2010-2014 Xilinx

XILINX

# PicoBlaze – Példányosítás



# PicoBlaze – Címzéstartomány

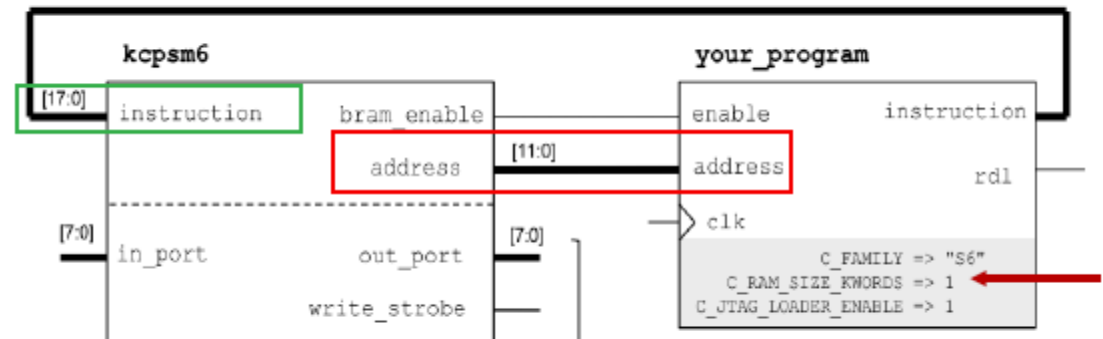
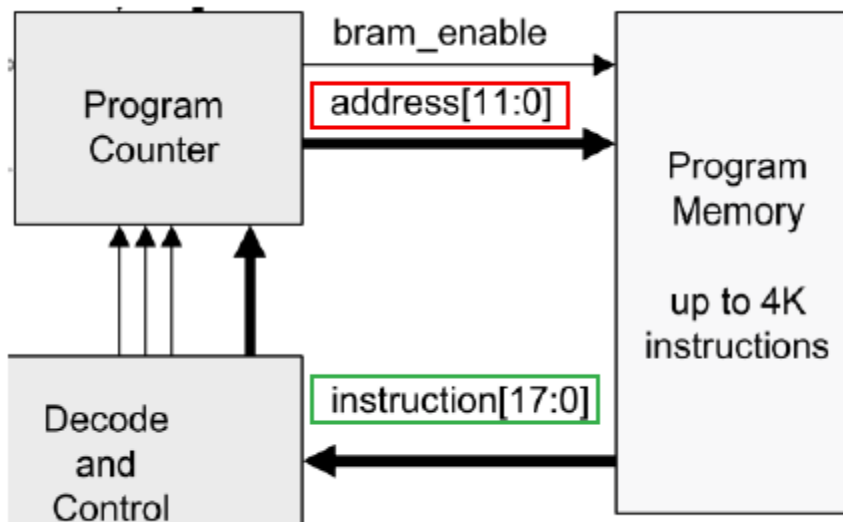
## Utasításmemória mérete

Az utasításmemória címe **12 bites**, így maximum **4K** méretű lehet, mivel  $2^{12} = 4K$ .

A választható utasításmemória-méretetek: **1K, 2K és 4K**.

Az utasítások **18 bit hosszúak**, ezért az utasításmemória méretei:

- ✓ **1K×18**
- ✓ **2K×18**
- ✓ **4K×18**



# PicoBlaze – Címzési módok

---

## Immediate mode

SUB s7, 0x07

ADDC s2, 0x08

$s7 \leq s7 - 0x07$

$s2 \leq s2 + 0x08 + C$

## Direct mode

ADD sa, sf

IN s5, 0x2a

$sa \leq sa + sf$

$s5 \leq \text{PORT}(0x2a)$

## Indirect mode

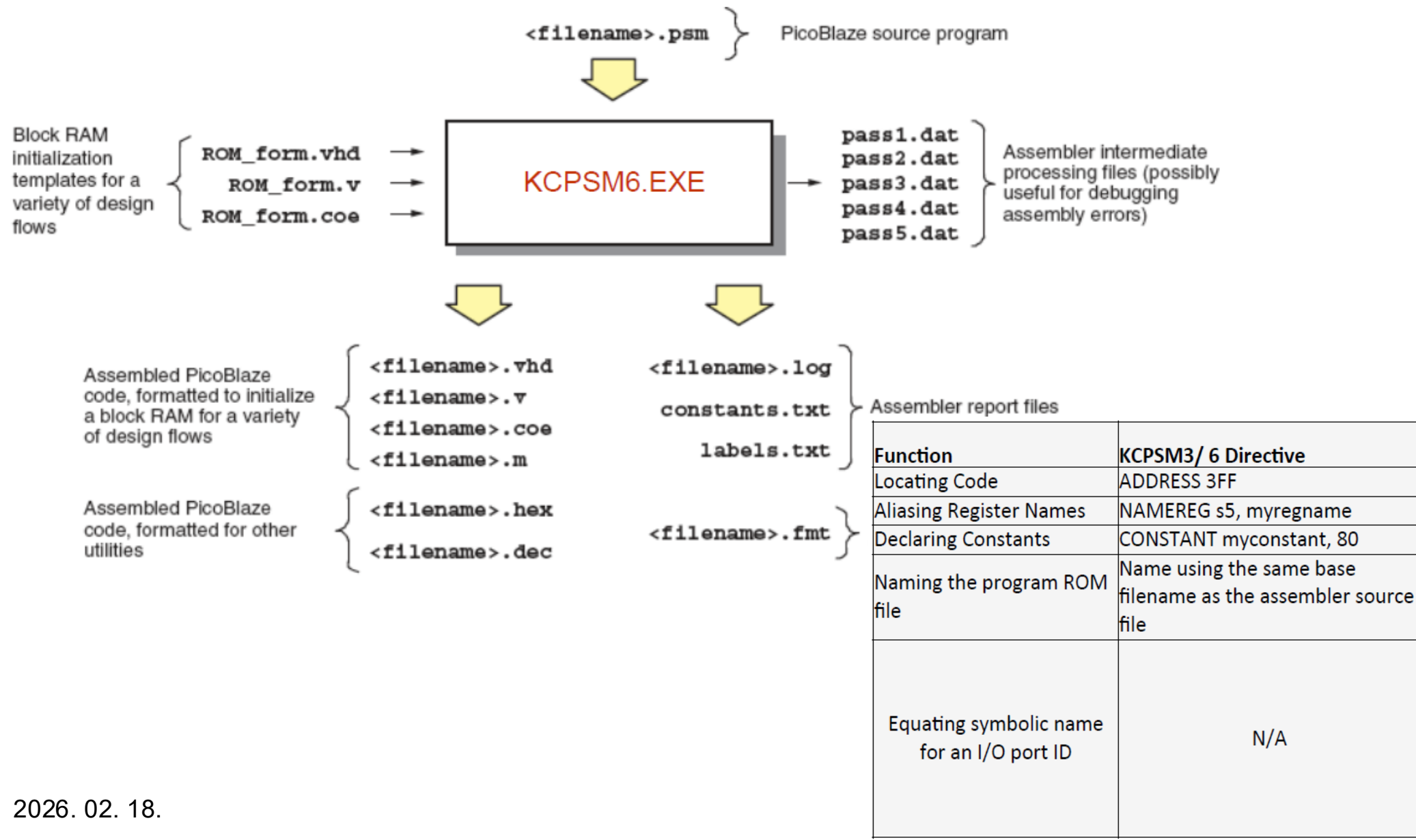
STORE s3, (sa)

IN s9, (s2)

$\text{RAM}((sa)) \leq s3$

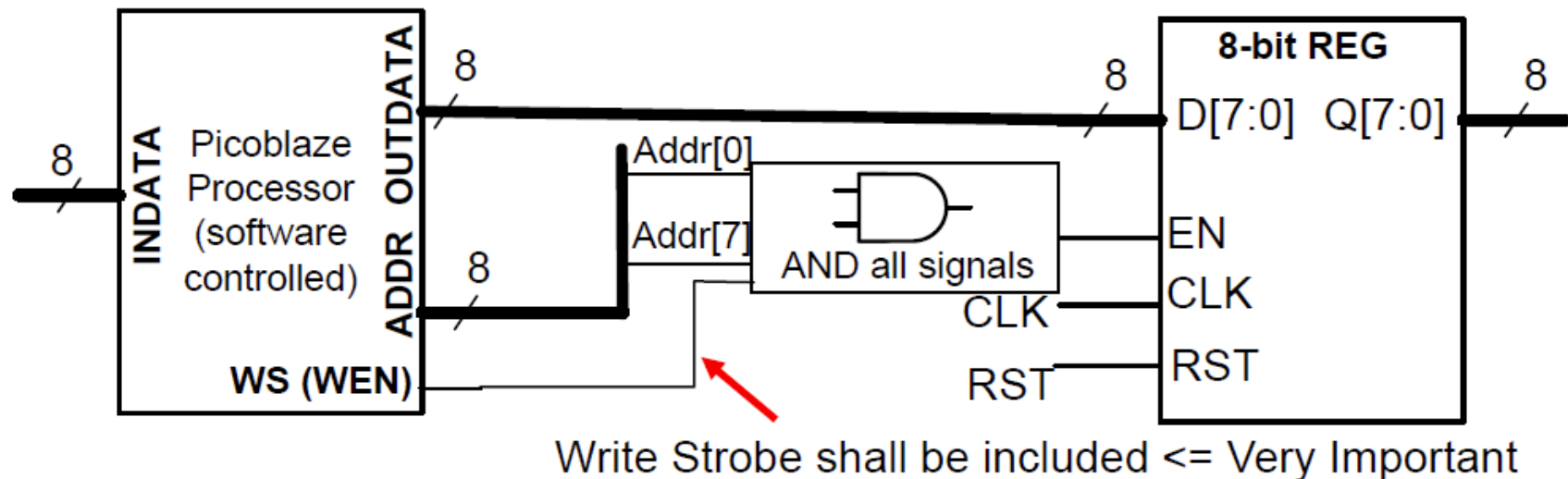
$s9 \leq \text{PORT}((s2))$

# PicoBlaze – Fordítás, Direktívák



# PicoBlaze – Gyakorlat – kimeneti port

Az alábbi regiszter rögzítse az adatot (**out\_data**) a **PicoBlaze**-ból, amikor az **FF hex** címre küld kimenetet (**cím vagy port azonosító** alapján).

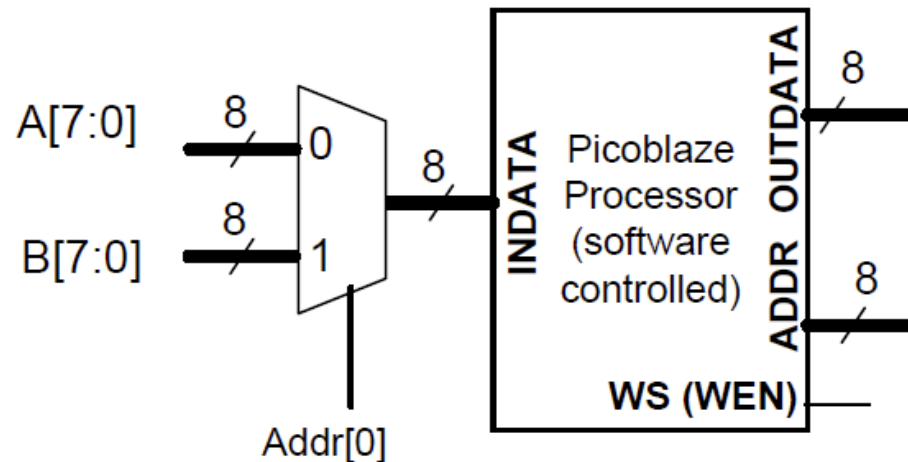


# PicoBlaze – Gyakorlat – bemeneti port

Használjuk a **PicoBlaze**-t, hogy bemenetet fogadjon:

- **A** a **00 hex** címről
- **B** a **0x01 hex** címről

Mivel az **INDATA** lábak dedikált bemeneti adatvonalak, figyelmen kívül hagyhatjuk a **READ STROBE** jelet káros következmények nélkül.



# PicoBlaze – Gyakorlat – memória címdekódolás

A címdekódolás arra a folyamatra utal, amely során a megfelelő eszközt engedélyezik egy adott címkombináció alapján.

